

MEMORIA ESCRITA DEL PROYECTO

CFGS Desarrollo de Aplicaciones Multiplataforma

VecindApp

Autor: Marius Ion Dodu

Tutor: Emilio Saurina

Fecha de entrega: 23/04/2026

Convocatoria: 2S2526

Documentos del proyecto:

https://drive.google.com/drive/folders/1qwgXD19ZrI29d8KI_CNAR5xNN_RNgBY?usp=sharing



Índice

Contenido

1. Introducción	6
1.1. Motivación	6
1.2. Abstract	6
1.3. Objetivos propuestos (generales y específicos).....	7
2. Estado del Arte	8
2.1. Contexto socioeconómico: Aislamiento urbano y economía colaborativa	8
2.2. Análisis competitivo del mercado.....	9
2.3. Necesidad de la accesibilidad universal.....	10
2.4. Nicho de mercado identificado.....	11
3. Metodología usada	11
4. Tecnologías y herramientas utilizadas en el proyecto	13
4.1. Tecnologías principales	13
4.2. Tecnologías de terceros	15
4.3. Herramientas de desarrollo y documentación	15
5. Planificación, Diagnóstico y Contexto Laboral.....	16
5.1. Contexto laboral	16
5.2. Planificación: estimación vs. realidad	16
5.3. Análisis DAFO	18
6. Análisis del proyecto	19
6.1.1. Requisitos funcionales.....	19
6.1.2. Requisitos no funcionales	23
6.2. Diagrama de Casos de Uso.....	26
Listado de casos de uso	27
Especificaciones de los casos de uso	27
6.3. Diagrama Entidad-Relación	32
Entidades y atributos	32
Relaciones y cardinalidades	34

Multiplicidad de roles y normalización	35
Decisiones de diseño complementarias	35
6.4. Diagrama de clases	35
Capa Domain	36
Capa Data.....	36
Capa Application	37
Capa Presentation	37
Decisiones de diseño destacables	38
7. Diseño del proyecto	38
7.1. Concepción visual inicial.....	39
7.2. Paleta de colores y sistema visual	39
7.3. Pantallas principales.....	40
Login y Registro	40
Escaparate.....	41
Crear y Detalle de servicio	41
Transacciones	42
Valoración con pictogramas ARASAAC	42
Perfil e Historial.....	43
7.4. Decisiones de diseño destacables.....	43
8. Despliegue y pruebas	44
8.1. Despliegue de la aplicación	44
Generación del APK	45
Instalación en dispositivo	45
Entorno de verificación.....	45
8.2. Estrategia de pruebas.....	46
Integración continua del testing en el desarrollo.....	46
Testing manual vs. automatizado.....	47
Enfoque iterativo: el ejemplo de la valoración unidireccional.....	47

8.3. Pruebas funcionales sobre los casos de uso	48
8.4. Bugs detectados y resueltos	49
Clasificación por severidad	51
8.5. Prueba de humo funcional	51
8.6. Limitaciones del proceso de pruebas.....	53
9. Conclusiones	53
9.1. Objetivos alcanzados.....	53
9.2. Objetivos no alcanzados.....	54
9.3. Experiencia vivida y dificultades	55
10. Vías futuras.....	56
10.1. Funcionalidades descartadas del MVP	56
10.2. Mejoras técnicas y de arquitectura	57
10.3. Mejoras de accesibilidad	58
10.4. Funcionalidades nuevas surgidas durante el desarrollo.....	58
10.5. Evolución del alcance del proyecto.....	59
11. Bibliografía / Webgrafía.....	60
Estadísticas demográficas y socioeconómicas	60
Legislación y normativa de accesibilidad.....	60
Estudios sobre comunicación aumentativa y pictogramas ARASAAC.....	61
Software, frameworks y recursos de código abierto	61
Plataformas competidoras y contexto del sector	61
Contexto socioeconómico y aislamiento urbano.....	62
12. Anexos.....	62
12.1. Manual de Instalación.....	62
Requisitos previos.....	62
Opción A — Instalación directa del APK (la recomendada).....	62
Opción B — Compilación desde el código fuente.....	64
Resolución de problemas frecuentes	66
12.2. Manual de Usuario	66

Primer acceso a la aplicación.....	66
Registro de un nuevo vecino.....	67
Navegación principal: el BottomNav.....	67
Explorar el Escaparate.....	68
Publicar un servicio	68
Detalle de un servicio y acciones disponibles	69
Solicitar un servicio	69
Gestionar las transacciones activas	70
Completar una transacción	70
Valorar con pictogramas ARASAAC	71
Consultar el perfil	71
Consultar el historial	72
Uso del Text-to-Speech (TTS)	72
Cerrar sesión	73
12.3. Diagramas	74
Casos de uso.....	74
Diagrama Entidad-Relación	75
Diagrama de clases	76
12.4. Glosario	79

1. Introducción

1.1. Motivación

Seguro que a muchos de nosotros nos ha tocado hacer algún arreglo en casa, presentar documentación ante la administración pública, hacer arreglos al coche, explicar conceptos sobre las nuevas tecnologías a familiares y/o amigos. También hemos sido partícipes en ayudar a nuestros seres queridos cuando han necesitado de nuestra ayuda. Pero ¿Qué pasa cuando no tienes el conocimiento necesario para la realización de estas tareas "simples" o no tienes el tiempo necesario para aprender estos conocimientos desde cero?

Es por eso que he pensado en una forma de conectar con las personas que te rodean, que tienen un gran conocimiento por compartir y además quieren formar vínculos con la comunidad que les rodea. Creo una comunidad tiene la oportunidad de brindarse apoyo mutuo y poder compartir lazos con esas personas nos puede acercar un poco a no estar tan cerca y a la vez tan desconectados.

Recuerdo que, de pequeño, en mi país natal era muy común conocer a muchos de mis vecinos del barrio e ir a sus casas para buscar a mis amigos y salir a jugar a la calle. Recuerdo el barrio como "seguro" porque a pesar de no tener supervisión de nuestros padres directamente, siempre había alguna persona adulta por la calle que nos conocía a nosotros o a nuestros padres. Incluso por las mañanas y las salidas de del colegio al mediodía, íbamos juntos cada uno a nuestras casas e íbamos desviándonos cada uno hacia su casa sin que nuestros padres sintieran que estábamos desprotegidos.

Creo que la desconexión entre vecinos y la precariedad económica son dos problemas crecientes en los núcleos urbanos. Según el INE (2024), más del 28% de los hogares en España se encuentran en riesgo de pobreza o exclusión social. Esta situación genera una paradoja: existen necesidades básicas sin cubrir (reparaciones o mantenimientos del hogar, acompañamiento a mayores, apoyo con trámites digitales) y simultáneamente personas con tiempo disponible y habilidades para ofrecer esos servicios. Lo que falta es un canal que conecte a esas personas.

Aquí es donde VecindApp viene a intentar cubrir esas deficiencias y ofrecer una solución a esas personas que quieran ofrecer su tiempo a la comunidad.

1.2. Abstract

VecindApp is a native Android application designed as a hyperlocal time bank where neighbours exchange services using hours as the sole unit of value instead of money. Users can register with a unique profile, publish service offers categorised by type (home repairs, elderly companionship, administrative assistance, among others), browse a neighbourhood

showcase, and request services from other residents. The transaction system atomically validates the buyer's balance before transferring hours between users.

The project places universal accessibility at its core. It integrates Android's native Text-to-Speech (TTS) engine to read titles, descriptions and reviews aloud, assisting users with low vision, reading difficulties or limited literacy. Additionally, the traditional star-based rating system is replaced by ARASAAC pictograms, a nationally recognised standard for cognitive accessibility, reducing cognitive barriers in the feedback process. Service categories are also visually identified through dynamically assigned pictograms.

On the technical side, VecindApp is built entirely offline-first using Room/SQLite, follows the MVVM + Clean Architecture pattern recommended by Google, and leverages MPAndroidChart for grouped bar charts displaying the user's earned-versus-spent hour history. A reactive notification badge informs users of transactions requiring their attention. The competitive analysis conducted during this project found no existing platform that combines hyperlocal social networking, time banking, and advanced accessibility features within a single application.

1.3. Objetivos propuestos (generales y específicos)

Objetivo General: Desarrollar y desplegar (en un entorno local) una aplicación móvil Android que sea plenamente funcional que permita a los usuarios gestionar, publicar e intercambiar servicios vecinales a través de un sistema de saldo de horas, garantizando en todo el proceso que la interfaz sea accesible para cualquier persona que la necesite.

Se aplicará el patrón de arquitectura MVVM junto con principios de Clean Architecture para asegurar un código escalable y mantenible. La verificación del correcto funcionamiento se realizará en un dispositivo móvil conectado mediante cable USB al ordenador.

Objetivos Específicos (SMART):

ID	Objetivo	Plazo	Criterio de éxito
OE-01	Diseñar e implementar una base de datos relacional local con SQLite/Room para la gestión de usuarios, servicios, transacciones y valoraciones.	Sprint 2	4 entidades con relaciones FK verificadas y seed de datos iniciales funcional
OE-02	Implementar un sistema de transacciones atómico que valide el saldo disponible y actualice automáticamente las horas de comprador y vendedor.	Sprint 3	Transacción completa sin estado inconsistente; saldo insuficiente rechazado
OE-03	Implementar TTS nativo de Android para lectura en voz alta de títulos, descripciones y valoraciones.	Sprint 5	Lectura funcional en español con botón de activación manual en las pantallas de detalle
OE-04	Integrar un sistema de valoraciones con pictogramas ARASAAC que sustituya las valoraciones numéricas clásicas.	Sprint 4	Selección múltiple de pictogramas con comentario opcional, guardada en BD y

			consultable desde el perfil
OE-05	Generar un dashboard gráfico del historial de horas con MPAndroidChart.	Sprint 3	Gráfico de barras agrupadas (ganadas vs. gastadas) funcional en la pestaña Historial
OE-06	Implementar un sistema de autenticación local con registro de usuarios únicos e inicio de sesión por selección de perfil.	Sprint 4–5	Registro con validación de nombre único; login funcional con persistencia de sesión
OE-07	Categorizar los servicios del escaparate mediante pictogramas dinámicos asignados por categoría.	Sprint 6	Pictogramas ARASAAC visibles en escaparate, perfil y detalle, gestionados por un mapper centralizado
OE-08	Implementar un sistema de notificaciones reactivo que informe al usuario de transacciones pendientes de atención.	Sprint 6	Badge numérico en el BottomNav, reactivo al cambio de usuario y limpio al cerrar sesión

Tabla 1. Objetivos SMART. Elaboración propia.

2. Estado del Arte

El siguiente capítulo analiza el panorama actual de las plataformas de economía colaborativa vecinal, los bancos de tiempo digitales y el estado de la accesibilidad universal. El objetivo es identificar el nicho que VecindApp pretende cubrir.

2.1. Contexto socioeconómico: Aislamiento urbano y economía colaborativa

El fenómeno del aislamiento social en las sociedades modernas ha alcanzado dimensiones que los especialistas catalogan como una crisis silenciosa de salud pública. Millones de personas sufren de soledad no deseada, impactando a adultos mayores, familias en transición y jóvenes inmersos en un entorno de hiperconectividad digital, pero con una gran desconexión física.

Este aislamiento no solo erosiona el tejido social, sino que tiene un impacto directo y documentado sobre la salud física y mental de las personas, aumentando la ansiedad, depresión y problemas cardiovasculares. Los perfiles más vulnerables son las personas adultas mayores, quienes al vivir solos enfrentan mayores niveles de estrés y deterioro cognitivo debido a la falta de interacción y de un sentido de pertenencia comunal.

En cuanto a la tecnología, que a menudo se percibe como un factor que fomenta el aislamiento, también se presenta como una herramienta clave para evitarlo. Diferentes comunidades están demostrando que el uso de plataformas digitales hiperlocales permite organizar redes vecinales de apoyo, fomentando la cooperación ciudadana y el voluntariado

en los proyectos intergeneracionales. Fortalecer la identidad del barrio ayuda a reducir la sensación de anonimato en la gran ciudad, transformándola en un entorno más humano, solidario y conectado.

Para hacer frente a esto, dentro de la economía solidaria, los bancos de tiempo surgen como mecanismo donde la unión de las personas y la colaboración mútua son la base para evitar ese aislamiento. En lugar de usar dinero, las personas intercambian su tiempo: una hora de ayuda vale lo mismo para todos, sin importar si se trata de arreglar algo en casa, enseñar un idioma, o acompañar a alguien en su día. La idea es sencilla: el tiempo de cada persona tiene el mismo valor y gracias a ese intercambio se pueden crear redes de apoyo entre vecinos y comunidades.

2.2. Análisis competitivo del mercado

Al analizar las herramientas tecnológicas que intentan mitigar esta situación, podemos encontrar diferentes enfoques con carencias significativas:

- **¿Tienes sal?:** Es el intento más potente de crear una red social de barrio en España. Fue el mayor exponente de este tipo de aplicaciones inspirada en la red alemana *Nebenan.de*. Llegó a activarse en más de 560 vecindarios entre Madrid, Barcelona y Valencia. Tenía un sistema de verificación que protegía el contenido de buscadores externos para evitar que grandes empresas hicieran acciones publicitarias de poco valor para la comunidad. Sin embargo, cesó sus operaciones permanentemente por falta de presupuesto evidenciando así la dificultad de monetización ética en redes hiperlocales.
- **Nexdoor:** Originada en San Francisco en 2011. En contraposición Nextdoor, ha consolidado un dominio global con presencia en 11 países y más de 265.000 barrios, gracias a su estrategia de verificación de residencia y su integración con organismos públicos y servicios de emergencia. Durante el COVID-19 demostró su gran valor sociológico a través de iniciativas contra la soledad no deseada facilitando videollamadas y apoyo logístico directo entre residentes para la distribución de recursos de primera necesidad. No obstante, opera bajo un modelo comercial respaldado por capital de riesgo, dependiendo de publicidad hipersegmentada, y no funciona como un banco de tiempo estructurado.
- **TimeOverflow:** En el nicho de los bancos de tiempo en España, TimeOverflow destaca como el software oficial adoptado por distintas asocia y recomendado por la ADBDT (Asociación para el Desarrollo de los Bancos de Tiempo). Desarrollado por la cooperativa tecnológica Coopdevs, permite publicar ofertas y demandas de servicios y gestionar cheques de tiempo digitales. Sin embargo, su concepción se asemeja más

a un portal de gestión administrativa que a una aplicación móvil con experiencia de usuario moderna, dejando un vacío para una app móvil moderna que ofrece una experiencia de usuario ágil.

- **TimeRepublik y otras plataformas intenacionales:** A nivel internacional, TimeRepublik opera como una red global de intercambio de tiempo con más de 100.000 usuarios en más de 100 países. Utilizan los “TimeCoins” como bloques de 15 minutos. Es un modelo que demuestra escalabilidad, pero pierde el concepto de hiperlocal y la confianza presencial que caracteriza al apoyo vecinal genuino.

Plataforma	Enfoque	Modelo	Accesibilidad	Estado (Es)
Nextdoor	Red social de barrio	Comercial / VC	WCAG básico	Operativa
¿Tienes sal?	Comunidad vecinal	Social / sin monetización	Básico	Cerrada
TimeOverflow	Gestión Bancos de Tiempo	Open source/ subvenciones	Estándar Web	Activa
TimeRepublik	Intercambio Global	Comercial (Time coins)	Estándar	Operativa
VecdinApp	Banco de Tiempo + red vecinal	Local/TFG	TTS + ARASAAC	En desarrollo

Tabla 2. Comparativa de plataformas vecinales y de banco de tiempo. Elaboración propia.

2.3. Necesidad de la accesibilidad universal

La OMS estima que aproximadamente el 15% de la población mundial (más de 1.000 millones de personas) convive con alguna forma de discapacidad. En cuanto a al año transcurrido (2025) y el año en curso (2026), la accesibilidad digital ha pasado de la categoría de buena práctica a ser de categoría obligatoria por ley. El Acta Europea de Accesibilidad (EAA), vigente desde junio de 2025, obliga a que los servicios digitales optimicen sus plataformas para garantizar la inclusión. El estándar de referencia es la versión 2.2 de las WCAG, y el Nivel AA es la línea base exigida. VecindApp aborda este reto mediante dos vectores que ninguna plataforma vecinal integra conjuntamente:

- **Text-to-Speech (TTS) nativo:** A diferencia de la compatibilidad pasiva con TalkBack (el lector de pantalla del sistema), VecindApp integrará un botón de lectura en voz alta dentro de la propia interfaz mediante la API TextToSpeech del SDK de Android. Esto es diferencial debido a que por ejemplo un adulto mayor con baja visión o una persona con dislexia no necesariamente sabe configurar TalkBack, pero sí puede pulsar un botón de «escuchar» claramente visible. Se pasa así de accesibilidad pasiva a accesibilidad activa.
- **Pictogramas ARASAAC para valoraciones:** El Centro Aragonés de la Comunicación Aumentativa y Alternativa (ARASAAC) ha desarrollado un sistema pictográfico con más de 12.000 pictogramas bajo licencia Creative Commons (BY-NC-

SA), consolidado como estándar global en Comunicación Aumentativa y Alternativa. Estudios publicados en *Frontiers in Psychology* demuestran que los pictogramas ARASAAC poseen un nivel de iconicidad (similitud visual con el concepto que representan) superior a sistemas comerciales como SPC o Bliss. La propuesta de VecindApp de utilizar estos pictogramas como sistema de valoración (sustituyendo las típicas estrellas numéricas por conceptos visuales como “amabilidad”, “puntualidad” o “trabajo bien hecho”) constituiría, según la investigación realizada, el primer uso documentado de pictogramas AAC en una plataforma de economía colaborativa a nivel mundial.

2.4. Nicho de mercado identificado

Tras la investigación hasta la fecha del panorama actual, se puede confirmar que la implementación de las tres funcionalidades de que venimos hablando sobre VecindApp (red social hiperlocal, banco de tiempo con horas como moneda y accesibilidad universal avanzada) está completamente vacía en el mercado global. Incluso la combinación de solo dos de las tres funcionalidades es bastante rara. Ninguna plataforma del banco de tiempo existente incorpora pictogramas, interfaces en lectura fácil ni diseño específico para personas con discapacidad cognitiva. Del mismo modo, ninguna aplicación vecinal integra TTS nativo (solo compatibilidad pasiva con lectores de pantalla del sistema).

Nota: Existe una startup colombiano llamada “Vecindapp” (<https://vecindapp.com.co/>), pero se trata de una app de administración de copropiedades sin banco de tiempo ni accesibilidad cognitiva. El nombre coincide, pero el concepto es completamente distinto.

3. Metodología usada

Para la elección de la metodología he tomado como referencia lo aprendido en la asignatura de Entornos de Desarrollo, donde se estudian las metodologías que siguen la filosofía ágil. Estas metodologías se basan en la flexibilidad a lo largo del ciclo de vida del proyecto y en la capacidad de adaptarse a los cambios que vayan surgiendo durante el desarrollo.

Las metodologías ágiles son métodos de gestión que permiten adaptar el ciclo de vida del proyecto de manera iterativa e incremental: el producto se va liberando por partes, de forma periódica, y cada entrega supone un incremento de funcionalidad respecto de la anterior. Entre las más conocidas se encuentran Scrum, Kanban y Programación Extrema (XP). Scrum estructura el trabajo en sprints de duración fija (entre dos y cuatro semanas) con reuniones de revisión y retrospectiva al final de cada uno. Kanban, por su parte, se centra en la visualización del flujo de tareas mediante un tablero de columnas (pendiente, en curso, hecho) y permite una gestión más continua sin depender de periodos cerrados.

Caso VecindApp

Para el desarrollo de este proyecto se ha adoptado una metodología ágil basada en SCRUM adaptada a un desarrollo individual. Se ha descartado aplicar Scrum de manera ortodoxa porque ciertas tareas ligadas a este (reunión diaria, retrospectiva en equipo) carecen de sentido cuando el equipo es una sola persona. En su lugar, se tomaron los elementos que sí resultaban útiles para un trabajo de esta envergadura: sprints de duración fija con un objetivo concreto por iteración, un backlog de tareas priorizado y la posibilidad de adelantar trabajo de sprints futuros cuando las dependencias ya estaban resueltas. Para el seguimiento visual de las tareas dentro de cada sprint se utilizó un enfoque tipo Kanban, organizando las tareas en estados (pendiente, en curso, hecho) lo que permitía tener una vista clara de lo que quedaba por hacer en cada momento.

El desarrollo se ha organizado en 7 sprints semanales. Los 6 primeros estuvieron dedicados al desarrollo de la aplicación y el séptimo a la fase de documentación y preparación de la presentación. Cada iteración tiene un objetivo concreto, unas tareas definidas y un objetivo de entrega verificable. De esta manera se pueden validar las funcionalidades de forma progresiva y ajustar el rumbo del desarrollo en función del tiempo disponible.

En mi caso, como el desarrollo se ha ido alternando en dos máquinas distintas, he tenido que usar mucho git para tener en el punto en el que volvía al desarrollo y el diario para no perder el hilo entre los distintos turnos de trabajo.

Las pruebas se realizaron mediante caja negra sobre los casos de uso definidos en la fase de análisis, verificando que cada requisito funcional se cumplía según lo especificado. Además, se mantuvo un diario de desarrollo continuo donde se documentan los problemas técnicos encontrados y las soluciones aplicadas, lo que garantiza la trazabilidad de las decisiones tomadas a lo largo del proyecto.

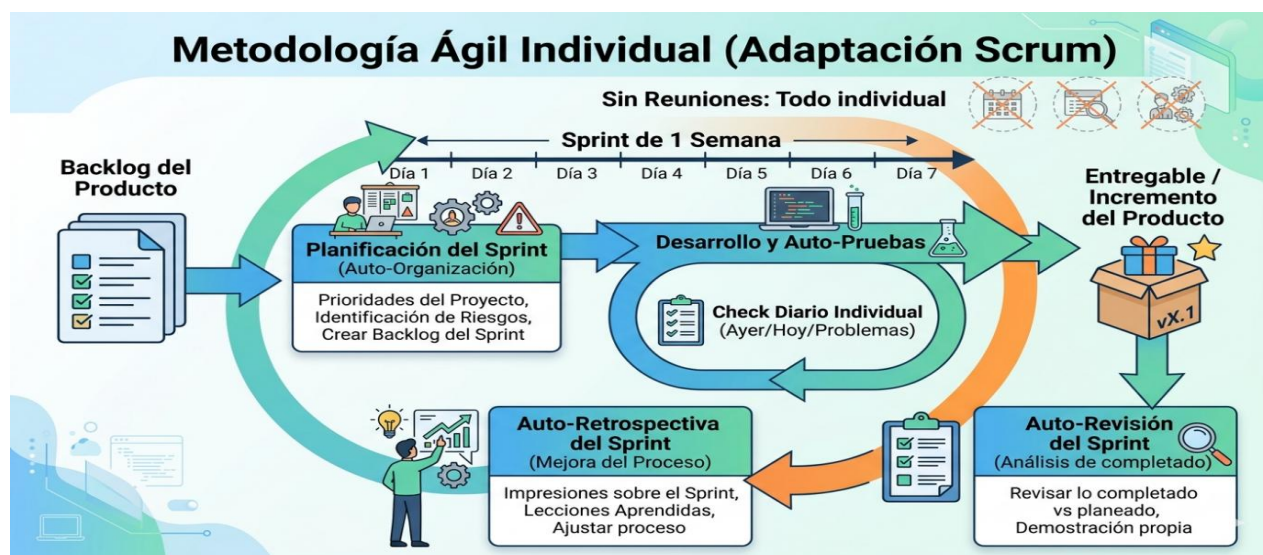


Figura 1. Adaptación Scrum. Imagen realizada con IA

Ciclo de vida del proyecto

El ciclo de vida se estructuró en las siguientes fases:

- **Fase 1 Diseño Funcional y Visual:** Definición de casos de uso, bocetos mediante herramientas de generación de imágenes de las pantallas principales y flujo de navegación de la aplicación.
- **Fase 2 Diseño Técnico:** Modelo de datos (diagrama E-R con 4 entidades), elección de la arquitectura MVVM + Clean Architecture y definición de la estructura de carpetas del proyecto.
- **Fase 3 Setup y Control de Versiones:** Creación del repositorio Git, configuración del proyecto en Android Studio con todas las dependencias y estrategia de ramas (desarrollo + sprints + ramas secundarias).
- **Fase 4 Desarrollo por Sprints:** 6 iteraciones semanales desde la estructura base y la base de datos hasta el pulido final de la interfaz. El MVP (producto mínimo viable) priorizó el login local, el catálogo de servicios, las transacciones de horas, las valoraciones con pictogramas ARASAAC y el TTS. Las funcionalidades secundarias (notificaciones reactivas, categorización visual, formateo de horas) se incorporaron en los sprints finales una vez que el núcleo funcional estaba estable.
- **Fase 5 Documentación y presentación:** Redacción de la memoria escrita, preparación de las diapositivas y grabación del vídeo de defensa del proyecto.

4. Tecnologías y herramientas utilizadas en el proyecto

(2-3 páginas)

A continuación, se detallan las tecnologías empleadas en el desarrollo de VecindApp, diferenciando entre las herramientas del ecosistema Android estándar (trabajadas durante el CFGS) y las librerías de terceros justificadas en la propuesta aprobada.

4.1. Tecnologías principales

Tecnología	Versión	Justificación
Kotlin	2.0.21 (octubre 2024)	Lenguaje oficial de Android recomendado por Google. Trabajado durante el CFGS.
Android Studio	Panda 3 2025.3.2 (marzo 2026)	IDE oficial para desarrollo Android. Incluye emulador, depurador y herramientas de layouts.
Room (SQLite)	2.8.4 (noviembre 2025)	Capa de abstracción sobre SQLite recomendada por Google que permite acceder a la base de datos sin problemas y, al mismo tiempo permite

		verificación de consultas SQL en tiempo de compilación y anotaciones que minimizan el código repetitivo.
ViewModel + LiveData	2.10.0 (marzo 2026)	Componentes de Android Jetpack para separar la lógica de negocio de la UI, respetando el ciclo de vida de los Fragments.
Navigation Component	2.9.7 (marzo 2026)	Navigation es un framework para navegar entre "destinos" dentro de una aplicación para Android que proporciona una API coherente, ya sea que los destinos se implementen como Fragment, Activity o algún otro tipo de componente.
TextToSpeech	SDK nativo	API nativa de Android para síntesis de voz. No requiere internet ni dependencias externas.
Kotlin Coroutines + StateFlow	1.9.0	Programación asíncrona estructurada. Se utilizan para las operaciones de base de datos en los repositorios y para los flujos reactivos en los ViewModels (flatMapLatest, stateIn).
Material Design Components	1.12.0	Librería oficial de Google que proporciona los componentes de interfaz utilizados en la app: BottomNavigationView, BottomSheet, Slider, Snackbar, entre otros.
KSP (Kotlin Symbol Processing)	2.0.21-1.0.28	Procesador de anotaciones moderno que sustituye a KAPT. Necesario para que Room genere el código de los DAOs en tiempo de compilación. Más rápido que su predecesor.

Tabla 3. Tecnologías principales del proyecto. Elaboración propia.

Considero necesario aclarar por qué el proyecto no utiliza ciertas librerías que forman parte del stack habitual en proyectos Android profesionales, como Hilt/Dagger para inyección de dependencias, Retrofit para comunicación con APIs REST o Glide para carga de imágenes. En primer lugar, VecindApp es una aplicación offline-first cuya totalidad de datos reside en una base de datos local SQLite gestionada con Room. No existe un servidor remoto ni una API REST con la que comunicarse, por lo que Retrofit carece de propósito en este proyecto. Del mismo modo, todas las imágenes utilizadas (pictogramas ARASAAC e iconos de categoría) están empaquetadas como assets locales en la carpeta /res/drawable, lo que hace innecesaria una librería de carga de imágenes remotas como Glide.

En cuanto a la inyección de dependencias, se optó por una inyección manual a través de la clase VecindAppApplication, que actúa como contenedor Singleton proporcionando la instancia de la base de datos y los cuatro repositorios. Esta decisión responde a dos motivos: por un lado, Hilt y Dagger son frameworks de terceros que no se han trabajado durante el CFGS y cuyo uso requiere justificación en la propuesta; por otro, la envergadura del proyecto

(4 repositorios, sin módulos Gradle adicionales) no justifica la complejidad añadida que introducen estas librerías. La solución manual implementada cumple el mismo objetivo (evitar instancias duplicadas de la base de datos, respetar el patrón Singleton) con una fracción del código de configuración. Para un proyecto de mayor envergadura o en un entorno de producción con múltiples módulos, la migración a Hilt sería el siguiente paso lógico y se contempla como vía futura.

4.2. Tecnologías de terceros

Las siguientes tecnologías de terceros han sido justificadas en la propuesta aprobada por el tutor:

Librería	Justificación de uso
ARASAAC (pictogramas)	Empaquetados como assets locales en formato PNG dentro de /res/drawable. Estándar nacional en accesibilidad cognitiva con licencia Creative Commons BY-NC-SA. Se utilizan tanto en el sistema de valoraciones (sustituyendo las clásicas estrellas numéricas) como para la identificación visual de las categorías de servicio en el escaparate.
MPAndroidChart	El SDK nativo de Android no incluye componentes de visualización de gráficos. Esta librería (v3.1.0) permite generar el gráfico de barras agrupadas del historial de horas ganadas vs. gastadas en la pestaña Historial.
Firestore	Implementación opcional para recolección anónima de métricas de uso (número de usuarios, horas intercambiadas). Excluida del MVP, contemplada en Vías Futuras.

Tabla 4. Tecnologías de terceros. Elaboración propia.

4.3. Herramientas de desarrollo y documentación

- **Git / GitHub:** Control de versiones con estrategia de ramas (main, desarrollo, sprint/, bugfix/) y commits bajo el estándar Conventional Commits.
- **diagrams.net (draw.io):** Creación de los diagramas UML (Casos de Uso, Entidad-Relación, Clases)
- **Mermaid:** Generación del diagrama de clases mediante sintaxis declarativa renderizada como gráfico.
- **Dispositivo físico Android:** Verificación y pruebas de la aplicación en un dispositivo móvil conectado al ordenador mediante cable USB.
- **Microsoft Word (Office 365):** Redacción y maquetación de la memoria escrita del proyecto.

- **Microsoft PowerPoint (Office 365):** Elaboración de las diapositivas para la presentación y defensa del proyecto.
- **VS Code:** Edición y mantenimiento del diario de desarrollo (memorias.txt), aprovechando la corrección ortográfica y el autocompletado.
- **NotebookLM:** Organización y consulta de los historiales de conversaciones de desarrollo mantenidos a lo largo del proyecto.
- **OBS Studio:** Para la grabación y edición del video de la defensa.

5. Planificación, Diagnóstico y Contexto Laboral

5.1. Contexto laboral

Este proyecto se ha desarrollado compaginando el CFGS de Desarrollo de Aplicaciones Multiplataforma con una jornada laboral completa. Esto ha hecho que la planificación se vea afectada de dos maneras: por un lado, las sesiones de desarrollo se repartían entre horas previas a la jornada laboral, pausas y tardes/noches; por otro, el entorno corporativo del puesto de trabajo (firewall Zscaler) introdujo dificultades técnicas adicionales a la hora de sincronizar dependencias con Gradle, que tuvieron que resolverse manualmente. Ambos factores se han tenido en cuenta a la hora de planificar los sprints y priorizar las funcionalidades del MVP.

5.2. Planificación: estimación vs. realidad

La planificación inicial contemplaba sprints semanales de duración uniforme. En la práctica, la duración real de cada sprint varió en función de la complejidad de las tareas y de la disponibilidad de tiempo. A continuación, se muestra la tabla de planificación con las fechas reales extraídas del historial de Git del proyecto:

Sprint	Contenido	Estimación	Fechas reales (Git)	Duración real	Observaciones
Sprint 1	Diseño funcional, diagramas UML, setup del proyecto, migración a KSP, estructura Clean Architecture	1 semana	06.03 – 16.03	11 días	Incluye la entrega de feedback al tutor (13-16 mar)
Sprint 2	BBDD Room (entidades + DAOs), CRUD de servicios, Escaparate con RecyclerView	1 semana	17.03 – 19.03	3 días	Se completó antes de lo previsto. La preparación en el S1 aceleró la implementación.
Sprint 3	Transacciones de horas, pantalla de perfil, historial con MPAndroidChart	1 semana	19.03 – 20.03	2 días	Finalizado en 2 días. Se adelantó trabajo previsto para sprints posteriores (historial y estadísticas) al

					tener las dependencias ya resueltas.
Sprint 4	Login/Registro con sesión local, sistema de valoraciones con pictogramas ARASAAC	1 semana	21.03 – 29.03	9 días	Se dedicó más tiempo a las valoraciones con pictogramas y a la corrección de bugs detectados en las pruebas funcionales del Sprint 3.
Sprint 5	TTS (accesibilidad), usuarios únicos, Spinner de barrios, pestañas en historial	1 semana	29.03 – 06.04	9 días	Implementación del TTS en todas las pantallas y pruebas funcionales extensivas
Sprint 6	Badge reactivo de notificaciones, pictogramas dinámicos por categoría, formateo de horas, pulido UI/UX	1 semana	07.04 – 13.04	7 días	Último sprint de desarrollo. Se priorizaron las notificaciones reactivas y el pulido visual sobre el WorkManager, que se descartó por falta de tiempo
Sprint 7	Memoria escrita, diapositivas, vídeo de defensa	1 semana	14.04 – 24.04	11 días	Fase ampliada al haberse liberado tiempo de los sprints anteriores.

Tabla 5. Planificación estimada vs. real del proyecto. Elaboración propia

La desviación más significativa fue la aceleración de los Sprints 2 y 3, que se completaron en menos de una semana cuando estaban estimados para dos semanas entre ambos. Esto se debe a que el trabajo de diseño técnico realizado en el Sprint 1 (diagramas E-R, de clases y estructura de carpetas) redujo la incertidumbre a la hora de codificar. Como consecuencia, esto me permitió avanzar a mayor velocidad en la implementación de la base de datos y las primeras pantallas. Siguiendo la flexibilidad que permite Scrum, se adelantó trabajo del backlog (historial y estadísticas, originalmente previstos para el Sprint 5) al Sprint 3 al tener ya resueltas las dependencias necesarias. En cambio, los Sprints 4 y 5 se extendieron ligeramente debido a la cantidad de bugs funcionales detectados durante las pruebas y a la complejidad de integrar el TTS de forma coherente en todas las pantallas de la aplicación. Por otra parte, el WorkManager para notificaciones locales programadas, contemplado originalmente en el Sprint 6, se descartó del alcance final por limitaciones de tiempo y se traslada a la sección de Vías Futuras.

VecindApp — Diagrama de Gantt

Planificación vs. Ejecución Real · TFG DAM · ILERNA · 2S2526

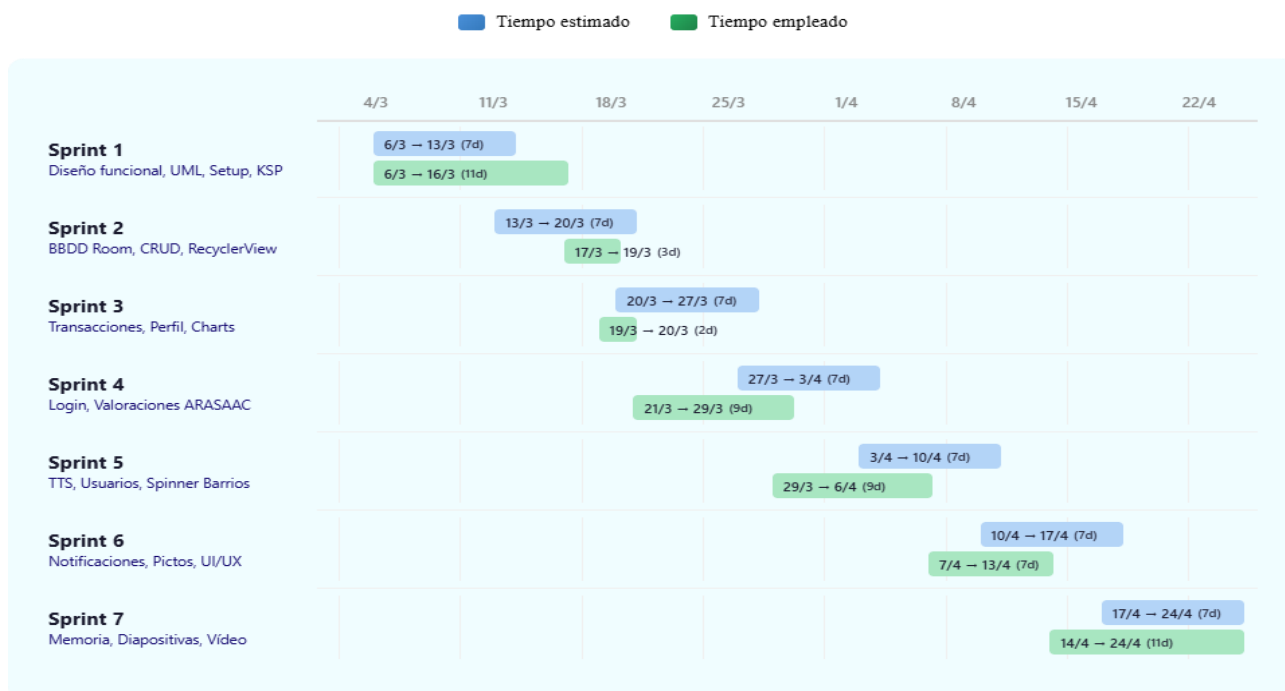


Diagrama de Gantt. Elaboración propia

5.3. Análisis DAFO

El análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) es una herramienta de diagnóstico estratégico que permite evaluar un proyecto o idea desde dos perspectivas: los factores internos (fortalezas y debilidades, propios del proyecto) y los factores externos (oportunidades y amenazas, procedentes del entorno). Se aplica a continuación a VecindApp para identificar sus puntos fuertes y los riesgos a tener en cuenta.

	Factores positivos	Factores negativos
Origen interno	Fortalezas: Nicho de mercado sin explotar (ninguna plataforma combina banco de tiempo + red vecinal + accesibilidad avanzada). Arquitectura MVVM + Clean Architecture que facilita el mantenimiento y la escalabilidad. Funcionamiento offline-first que elimina la dependencia de un servidor remoto y reduce costes de infraestructura. Doble vector de accesibilidad (TTS activo +	Debilidades: Desarrollo individual, sin equipo de revisión de código ni testing dedicado. Tiempo limitado por la compaginación con una jornada laboral completa. Ausencia de backend remoto, lo que impide la sincronización de datos entre dispositivos. Sin sistema de autenticación con contraseña real (login simplificado para facilitar la demo).

	pictogramas ARASAAC) que diferencia la app de toda la competencia analizada.	
Origen externo	Oportunidades: El Acta Europea de Accesibilidad (EAA), vigente desde junio de 2025, obliga a que los servicios digitales cumplan estándares de inclusión, lo que posiciona a VecindApp como una solución ya alineada con la normativa. El crecimiento de la economía colaborativa y el envejecimiento de la población generan una demanda creciente de redes de apoyo vecinal. Los pictogramas ARASAAC gozan de reconocimiento institucional y licencia abierta, lo que facilita su adopción.	Amenazas: Plataformas consolidadas como Nextdoor podrían incorporar funcionalidades de banco de tiempo en el futuro. La dificultad de monetización ética en redes hiperlocales (como demuestra el cierre de ¿Tienes sal?). La brecha digital en los perfiles de usuario objetivo (personas mayores) puede dificultar la adopción inicial de la aplicación.

Tabla 6. Análisis DAFO de VecindApp. Elaboración propia.

6. Análisis del proyecto

6.1. Requisitos funcionales y no funcionales

Esta sección recoge el análisis funcional completo de VecindApp: los requisitos funcionales y no funcionales, los diagramas UML obligatorios y las especificaciones de los casos de uso principales.

6.1.1. Requisitos funcionales

Los requisitos se clasifican en funcionales (qué debe hacer el sistema) y no funcionales (cómo debe comportarse). Se ha asignado además un nivel de prioridad a cada uno: Alta para los imprescindibles del MVP, Media para funcionalidades que aportan valor pero no bloquean el núcleo, y Baja para pulido y mejoras de usabilidad. Todos los requisitos listados se encuentran implementados en la versión final entregada.

RF	Módulo de autenticación	Prioridad	Estado
RF-01	El sistema permitirá registrar un nuevo vecino con nombre (único, case-insensitive) y barrio	Alta	Implementado

	seleccionado de un Spinner con los barrios de Zaragoza predefinidos.		
RF-02	El sistema permitirá iniciar sesión introduciendo el nombre del vecino previamente registrado, sin contraseña, para facilitar la demo multiusuario.	Alta	Implementado
RF-03	El saldo inicial de cada usuario nuevo será de 5 horas como crédito de bienvenida.	Alta	Implementado
RF-04	El sistema mantendrá la sesión del usuario entre ejecuciones mediante SharedPreferences, permitiéndole cerrarla manualmente desde el perfil.	Alta	Implementado

RF	Módulo de servicios	Prioridad	Estado
RF-10	El usuario podrá publicar una oferta de servicio con: título, descripción, categoría (Hogar, Tecnología, Educación, Compañía, Recados u Otros) y coste en horas mediante un Slider con pasos de 0,25 h.	Alta	Implementado
RF-11	La pantalla principal mostrará un Escaparate del Barrio con todas las ofertas activas, ordenadas por fecha de publicación descendente.	Alta	Implementado
RF-12	Cada servicio mostrará un pictograma ARASAAC asignado dinámicamente en función de su categoría mediante CategoriaMapper.	Media	Implementado
RF-13	El usuario podrá editar o eliminar sus propios servicios únicamente si están en estado ACTIVO (sin interesados).	Alta	Implementado

RF-14	El usuario podrá cancelar una solicitud sobre un servicio propio que ya haya sido reservado, devolviéndolo al estado ACTIVO.	Media	Implementado
RF-15	El sistema gestionará el ciclo de vida del servicio mediante los estados ACTIVO, RESERVADO, COMPLETADO y CADUCADO.	Alta	Implementado

RF	Módulo de transacciones	Prioridad	Estado
RF-20	Al solicitar un servicio, el sistema validará que el saldo del comprador sea $\geq$ al coste antes de crear la transacción, impidiendo bloquear servicios sin saldo suficiente.	Alta	Implementado
RF-21	Al completar la transacción, el sistema restará las horas del comprador, las añadirá al vendedor e incrementará los contadores de intercambios de ambos, todo de forma atómica.	Alta	Implementado
RF-22	El sistema registrará cada transacción con: timestamp, comprador, vendedor, servicio, horas transferidas y estado (PENDIENTE, ACEPTADA, COMPLETADA, CANCELADA).	Alta	Implementado
RF-23	La pestaña de Transacciones mostrará al usuario todas sus transacciones activas con su rol (comprador/vendedor) diferenciado visualmente.	Alta	Implementado
RF-24	El sistema impedirá que un usuario solicite su propio servicio.	Alta	Implementado

RF	Módulo de valoraciones (ARASAAC)	Prioridad	Estado
RF-30	El sistema mostrará un BottomSheet de valoración con 9 pictogramas ARASAAC seleccionables	Alta	Implementado

	(puntual, amable, cumple lo prometido, etc.) más un campo opcional de comentario.		
RF-31	Los pictogramas estarán empaquetados en /res/drawable como assets PNG, sin dependencia de internet.	Alta	Implementado
RF-32	Solo el comprador podrá valorar al vendedor tras completarse la transacción (regla unidireccional).	Alta	Implementado
RF-33	El sistema impedirá valoraciones duplicadas sobre una misma transacción.	Alta	Implementado
RF-34	El usuario podrá consultar sus valoraciones enviadas y recibidas mostrando el título apropiado según el rol.	Media	Implementado

RF	Módulo de perfil e historial	Prioridad	Estado
RF-40	El perfil mostrará el saldo actual de horas, el nivel del vecino (Novato, Activo, Veterano, Referente) y el número de intercambios completados.	Alta	Implementado
RF-41	El nivel se recalculará automáticamente según el número de intercambios: Novato (0-2), Activo (3-5), Veterano (6-8), Referente (9+).	Media	Implementado
RF-42	El perfil mostrará la lista de servicios publicados por el usuario con acceso directo a su detalle.	Alta	Implementado
RF-43	El Historial mostrará un gráfico de barras agrupadas (MPAndroidChart) con el balance de horas ganadas vs. gastadas.	Alta	Implementado
RF-44	El Historial contará con dos pestañas independientes para transacciones completadas y canceladas.	Media	Implementado

RF	Módulo de accesibilidad (TTS)	Prioridad	Estado
-----------	--------------------------------------	------------------	---------------

RF-50	El sistema dispondrá de un botón visible de lectura en voz alta (TTS nativo) en las pantallas de Escaparate, Detalle del Servicio, Perfil y BottomSheet de Valoración.	Alta	Implementado
RF-51	El TTS leerá los textos en español con un formato humanizado (ej.: "dos horas y media" en lugar de "2.5 horas").	Media	Implementado
RF-52	El TTS adaptará su contenido al contexto: en valoraciones distinguirá entre "enviada" y "recibida" según el rol del usuario.	Media	Implementado

RF	Módulo de notificaciones	Prioridad	Estado
RF-60	El BottomNav mostrará un badge numérico en la pestaña de Transacciones con el conteo de transacciones que requieren atención (pendientes + completadas sin valorar como comprador).	Media	Implementado
RF-61	El badge se actualizará de forma reactiva al cambiar de usuario (login/logout) sin necesidad de reiniciar la aplicación.	Media	Implementado

6.1.2. Requisitos no funcionales

Los requisitos no funcionales definen las cualidades que debe cumplir el sistema en su conjunto: cómo debe comportarse, qué características debe tener y bajo qué restricciones debe operar. Se agrupan por categorías para facilitar su verificación.

RNF	Accesibilidad	Prioridad
RNF-01	La aplicación integrará accesibilidad activa mediante un botón TTS visible, sin depender de que el usuario tenga configurado TalkBack u otros lectores de pantalla del sistema.	Alta

RNF-02	El sistema de valoración no empleará indicadores numéricos (estrellas), sino pictogramas ARASAAC de reconocimiento institucional para reducir las barreras cognitivas.	Alta
RNF-03	Los elementos interactivos mantendrán un tamaño mínimo apto para usuarios con movilidad reducida (botones grandes, áreas de tap amplias).	Media
RNF-04	Los pictogramas de categoría se asignarán automáticamente para ofrecer una identificación visual rápida, independientemente de la capacidad lectora del usuario.	Media

RNF	Usabilidad	Prioridad
RNF-05	La navegación principal se realizará mediante un BottomNavigationView de 4 pestañas (Escaparate, Transacciones, Historial, Perfil) accesible desde cualquier punto de la aplicación.	Alta
RNF-06	Las acciones destructivas (eliminar, cancelar, completar transacción) requerirán confirmación previa mediante un diálogo modal.	Alta
RNF-07	El feedback al usuario se realizará mediante Snackbars contextuales, evitando Toasts clásicos para mantener coherencia visual.	Media
RNF-08	El formato de horas mostrado al usuario será legible en lenguaje natural (ej.: "dos horas y media" en TTS, "2 h 30 min" en pantalla).	Media

RNF	Rendimiento	Prioridad
RNF-09	La aplicación funcionará en modo offline-first: todas las operaciones de lectura y escritura se realizarán sobre la base de datos local SQLite sin requerir conexión a Internet.	Alta
RNF-10	Las listas (Escaparate, Transacciones, Historial) se implementarán con RecyclerView y DiffUtil para evitar repintados completos y optimizar el rendimiento.	Alta

RNF-11	Las operaciones de base de datos se ejecutarán de forma asíncrona mediante corrutinas (viewModelScope) para no bloquear el hilo principal.	Alta
RNF-12	Los flujos reactivos (StateFlow, Flow) se suscribirán respetando el ciclo de vida del Fragment (repeatOnLifecycle) para liberar recursos cuando la pantalla no esté visible.	Alta

RNF	Seguridad e integridad de datos	Prioridad
RNF-13	Las transferencias de horas entre usuarios se ejecutarán como operaciones atómicas de Room (@Transaction) para garantizar que no queden estados inconsistentes en caso de fallo.	Alta
RNF-14	Las relaciones entre entidades se protegerán mediante restricciones de Foreign Key con política ON DELETE CASCADE donde proceda.	Alta
RNF-15	El sistema impedirá la creación de usuarios con nombres duplicados mediante un índice UNIQUE a nivel de base de datos.	Alta

RNF	Mantenibilidad y escalabilidad	Prioridad
RNF-16	El código seguirá el patrón arquitectónico MVVM combinado con los principios de Clean Architecture, separando las capas de datos, dominio y presentación.	Alta
RNF-17	Los repositorios se definirán como interfaces en la capa de dominio, con sus implementaciones en la capa de datos, para facilitar el testing y la sustitución de fuentes.	Alta
RNF-18	Todas las clases y funciones públicas dispondrán de documentación KDoc explicando su propósito, parámetros y retorno.	Media
RNF-19	El código se versionará mediante Git siguiendo la convención de mensajes Conventional Commits (feat, fix, refactor, chore, docs, build).	Media

RNF-20	Las operaciones de formato reutilizables (costes, horas, pluralización) se centralizarán en clases helper para evitar duplicación.	Media
--------	--	-------

RNF	Compatibilidad	Prioridad
RNF-21	La aplicación será compatible con dispositivos Android 8.0 (API 26) o superior, cubriendo más del 95% del parque de dispositivos activos.	Alta
RNF-22	La interfaz se desarrollará íntegramente con layouts XML tradicionales, sin emplear Jetpack Compose, en coherencia con la propuesta aprobada por el tutor.	Alta
RNF-23	La aplicación se probará y verificará en un dispositivo físico Android conectado al ordenador mediante cable USB.	Alta

RNF	Internacionalización y textos	Prioridad
RNF-24	Todos los textos visibles para el usuario se extraerán a /res/values/strings.xml, facilitando una futura traducción.	Media
RNF-25	El idioma por defecto de la aplicación será español (es-ES), incluyendo la síntesis de voz del TTS.	Alta

6.2. Diagrama de Casos de Uso

La aplicación cuenta con un único actor (Usuario/Vecino) que puede desempeñar dos roles dentro del sistema en función del contexto de la transacción: comprador (solicita un servicio y gasta horas) o vendedor (ofrece un servicio y gana horas). Ambos roles los ejerce la misma persona según el caso, por lo que el diagrama los representa bajo un actor unificado.

Se han identificado 12 casos de uso que cubren las funcionalidades principales de la aplicación. Se establecen las siguientes relaciones UML entre ellos:

- Relación <<extend>>: el CU-05 (Escuchar contenido con TTS) extiende a los CU-04 (Explorar escaparate), CU-08 (Ver detalle del servicio), CU-11 (Valorar) y CU-12 (Consultar perfil), ya que la lectura en voz alta es un comportamiento opcional que se activa desde cualquiera de estas pantallas al pulsar el botón correspondiente.

- Relación <<extend>>: el CU-11 (Valorar con pictogramas ARASAAC) extiende al CU-10 (Completar transacción) aplicando la regla unidireccional: únicamente el comprador puede invocar la valoración tras completarse la transacción, y de forma voluntaria.
- Relación <<include>>: el CU-10 (Completar transacción) incluye obligatoriamente la validación de saldo que también utiliza el CU-09 (Solicitar servicio), garantizando la integridad del intercambio.

Listado de casos de uso

ID	Caso de uso	Actor
CU-01	Registrarse como nuevo vecino	Usuario
CU-02	Iniciar sesión	Usuario
CU-03	Cerrar sesión	Usuario
CU-04	Explorar escaparate	Usuario
CU-05	Escuchar contenido (TTS)	Usuario
CU-06	Publicar servicio	Usuario (vendedor)
CU-07	Editar o eliminar servicio propio	Usuario (vendedor)
CU-08	Ver detalle del servicio	Usuario
CU-09	Solicitar servicio	Usuario (comprador)
CU-10	Completar transacción	Usuario (vendedor)
CU-11	Valorar con pictogramas ARASAAC	Usuario (comprador)
CU-12	Consultar perfil e historial	Usuario

Tabla 7. Listado de casos de uso de VecindApp. Elaboración propia.

Especificaciones de los casos de uso

Siguiendo el requerimiento normativo de incluir las especificaciones de cada caso de uso, se presenta a continuación la ficha detallada de los más relevantes. Los casos de uso de menor complejidad (CU-03, CU-05, CU-08, CU-12) se describen de forma resumida por tratarse de flujos lineales sin ramificaciones significativas.

VecindApp

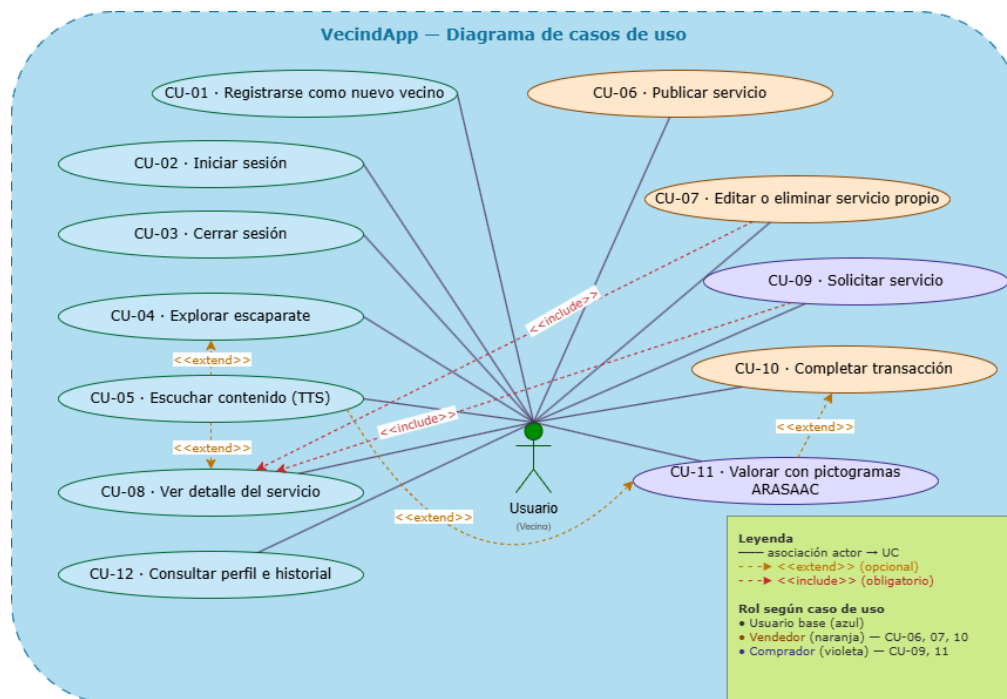


Diagrama de Casos de Uso de VecindApp. Elaboración propia en draw.io

CU-01. Registrarse como nuevo vecino

Actor	Usuario
Precondición	No existe sesión activa. El usuario se encuentra en la pantalla de Login.
Postcondición	Se crea el usuario en la BBDD con saldo inicial de 5 horas y nivel NOVATO. Se inicia sesión automáticamente.
Flujo principal	1. El usuario pulsa "Registrarse como nuevo vecino". 2. El sistema muestra el formulario de registro. 3. El usuario introduce un nombre y selecciona su barrio del Spinner. 4. El usuario pulsa "Registrarse". 5. El sistema valida que el nombre no esté duplicado. 6. El sistema inserta el usuario en Room y guarda su ID en SharedPreferences. 7. El sistema navega al Escaparate.
Flujo alternativo	5a. Si el nombre ya existe, el sistema muestra un Snackbar de error y permanece en la pantalla.

CU-02. Iniciar sesión

Actor	Usuario
Precondición	El usuario ya está registrado. No hay sesión activa.

Postcondición	La sesión queda guardada en SharedPreferences y se muestra el Escaparate.
Flujo principal	1. El usuario introduce su nombre en la pantalla de Login. 2. El usuario pulsa "Iniciar sesión". 3. El sistema busca al usuario en Room (case-insensitive). 4. El sistema guarda el ID del usuario en SharedPreferences. 5. El sistema navega al Escaparate.
Flujo alternativo	3a. Si el nombre no existe, el sistema muestra un Snackbar de error invitando al registro.

CU-04. Explorar escaparate

Actor	Usuario
Precondición	Hay sesión activa.
Postcondición	El usuario visualiza la lista de servicios ACTIVOS del barrio.
Flujo principal	1. El usuario accede a la pestaña Escaparate del BottomNav. 2. El sistema recupera todos los servicios en estado ACTIVO. 3. El sistema los muestra ordenados por fecha de publicación descendente. 4. Cada tarjeta muestra título, coste en horas y pictograma ARASAAC asignado por categoría.
Extensión	El usuario puede pulsar el FAB de TTS para escuchar todos los servicios visibles en pantalla (CU-05). O pulsación larga en cada tarjeta para escuchar TTS de esa tarjeta.

CU-06. Publicar servicio

Actor	Usuario (vendedor)
Precondición	Hay sesión activa.
Postcondición	Se crea un nuevo servicio en estado ACTIVO visible en el Escaparate.
Flujo principal	1. El usuario pulsa el FAB "+" del Escaparate. 2. El sistema muestra el formulario de creación. 3. El usuario introduce título, descripción, selecciona categoría y ajusta el coste mediante un Slider (pasos de 0,25 h). 4. El usuario pulsa "Guardar". 5. El sistema valida que el título no esté vacío. 6. El sistema inserta el servicio en Room. 7. El sistema navega de vuelta al Escaparate mostrando un Snackbar de confirmación.
Flujo alternativo	5a. Si el título está vacío, el sistema muestra un error y permanece en la pantalla.

CU-07. Editar o eliminar servicio propio

Actor	Usuario (vendedor)
Precondición	El usuario es el propietario del servicio. El servicio está en estado ACTIVO.
Postcondición	El servicio queda modificado o eliminado de la BBDD.
Flujo principal (editar)	1. El usuario accede al detalle del servicio desde el Escaparate o el Perfil. 2. El usuario pulsa "Editar". 3. El sistema abre el formulario con los datos precargados. 4. El usuario modifica los campos deseados y guarda. 5. El sistema actualiza el registro en Room.
Flujo principal (eliminar)	1. El usuario pulsa "Eliminar" desde el detalle. 2. El sistema muestra un diálogo de confirmación. 3. El usuario confirma. 4. El sistema elimina el servicio de Room (ON DELETE CASCADE también elimina transacciones asociadas).
Excepciones	Si el servicio está en estado RESERVADO o COMPLETADO, los botones de Editar/Eliminar no se muestran.

CU-09. Solicitar servicio

Actor	Usuario (comprador)
Precondición	Hay sesión activa. El servicio existe, está en estado ACTIVO y no es propiedad del usuario.
Postcondición	Se crea una transacción en estado PENDIENTE y el servicio pasa a estado RESERVADO.
Flujo principal	1. El usuario accede al detalle del servicio. 2. El usuario pulsa "Solicitar servicio". 3. El sistema valida que el saldo del comprador sea $\geq$ al coste del servicio. 4. El sistema muestra un diálogo de confirmación informando de las horas a transferir. 5. El usuario confirma. 6. El sistema crea la transacción (estado PENDIENTE) y cambia el servicio a RESERVADO. 7. El sistema muestra un Snackbar de éxito.
Flujo alternativo	3a. Si el saldo es insuficiente, el sistema muestra un error indicando las horas necesarias y no crea la transacción.
Excepciones	Si el usuario intenta solicitar su propio servicio, el sistema lo impide con un mensaje de error.

CU-10. Completar transacción

Actor	Usuario (vendedor)
Precondición	Existe una transacción en estado ACEPTADA donde el usuario es el vendedor.
Postcondición	La transacción pasa a COMPLETADA. Se transfieren las horas entre usuarios. El servicio pasa a COMPLETADO. Se incrementan los contadores de intercambios de ambos.
Flujo principal	1. El usuario accede a la pestaña Transacciones. 2. El usuario pulsa "Completar" en la tarjeta correspondiente. 3. El sistema muestra un diálogo de confirmación. 4. El usuario confirma. 5. El sistema ejecuta la operación atómica: - Debita horas al comprador. - Acredita horas al vendedor. - Incrementa el contador de intercambios de ambos y recalcula su nivel. - Cambia la transacción a COMPLETADA. - Cambia el servicio a COMPLETADO. 6. La UI se actualiza reactivamente.
Extensión	Tras completarse, el comprador puede invocar el CU-11 para valorar al vendedor.

CU-11. Valorar con pictogramas ARASAAC

Actor	Usuario (comprador)
Precondición	Existe una transacción en estado COMPLETADA donde el usuario es el comprador. No se ha valorado previamente.
Postcondición	Se crea una valoración asociada a la transacción con los pictogramas seleccionados y un comentario opcional.
Flujo principal	1. El usuario pulsa "Valorar" en la tarjeta de la transacción completada. 2. El sistema abre el BottomSheet de valoración con 9 pictogramas ARASAAC. 3. El usuario selecciona uno o varios pictogramas. 4. (Opcional) El usuario escribe un comentario en el campo de texto. 5. El usuario pulsa "Enviar valoración". 6. El sistema valida que se haya seleccionado al menos un pictograma. 7. El sistema inserta la valoración en Room. 8. El BottomSheet se cierra y la lista de transacciones se recarga para ocultar el botón "Valorar".
Flujo alternativo	6a. Si no hay ningún pictograma seleccionado, el sistema muestra un mensaje y no cierra el BottomSheet.

Extensión	El usuario puede pulsar el botón TTS para escuchar los pictogramas seleccionados y el comentario (CU-05).
------------------	---

Casos de uso de flujo simple

CU-03. Cerrar sesión: Desde el Perfil, el usuario pulsa "Cerrar sesión". El sistema borra el ID de SharedPreferences, limpia el badge de notificaciones y navega al Login.

CU-05. Escuchar contenido (TTS): En cualquiera de las pantallas con botón de altavoz (Escaparate, Detalle, Perfil, Valoración), el usuario pulsa el botón y el sistema reproduce por voz el contenido contextual (servicios visibles, descripción del servicio, saldo y nivel, o pictogramas seleccionados). La reproducción se detiene automáticamente al cambiar de pantalla.

CU-08. Ver detalle del servicio: El usuario pulsa cualquier tarjeta de servicio en el Escaparate, Perfil o Transacciones. El sistema muestra la información completa del servicio y los botones de acción disponibles según el rol del usuario y el estado del servicio.

CU-12. Consultar perfil e historial: El usuario accede a las pestañas Perfil o Historial. El perfil muestra saldo, nivel, intercambios totales y lista de servicios publicados. El historial muestra un gráfico de barras (MPAndroidChart) con el balance ganadas/gastadas y dos pestañas (completadas/canceladas) con el listado de transacciones.

6.3. Diagrama Entidad-Relación

La base de datos local SQLite se gestiona mediante Room y se compone de 4 entidades principales, cumpliendo con creces el mínimo normativo de 3 tablas relacionadas. Todas las entidades están relacionadas mediante claves foráneas declaradas explícitamente en el esquema Room, con política ON DELETE CASCADE para garantizar la integridad referencial en caso de borrado.

Entidades y atributos

USUARIO (tabla usuario): representa a un vecino registrado en la comunidad. Dispone de un índice único sobre el campo *nombre* para impedir duplicados.

Atributo	Tipo	Descripción
id_usuario (PK)	INTEGER	Clave primaria autoincremental
nombre	TEXT (UNIQUE)	Nombre visible del vecino
barrio	TEXT	Barrio de Zaragoza (enum)

avatar_path	TEXT (nullable)	Ruta local de la imagen de perfil
saldo_horas	REAL	Horas disponibles (inicial: 5.0)
intercambios_total	INTEGER	Número acumulado de transacciones completadas
nivel	TEXT	Nivel del vecino (enum: NOVATO, ACTIVO, VETERANO, REFERENTE)
fecha_registro	INTEGER	Timestamp del alta

SERVICIO (tabla servicio): representa un servicio publicado en el escaparate.

Atributo	Tipo	Descripción
id_servicio (PK)	INTEGER	Clave primaria autoincremental
id_usuario_fk (FK)	INTEGER	FK → usuario.id_usuario
titulo	TEXT	Título del servicio
descripcion	TEXT (nullable)	Descripción detallada
categoria	TEXT	Categoría del servicio (enum)
coste_horas	REAL	Coste en horas del servicio
estado	TEXT	Estado del ciclo de vida (enum: ACTIVO, RESERVADO, COMPLETADO, CADUCADO)
fecha_publicacion	INTEGER	Timestamp de publicación
fecha_caducidad	INTEGER (nullable)	Timestamp de expiración opcional

TRANSACCION (tabla transaccion): representa un intercambio de horas entre dos vecinos.

Atributo	Tipo	Descripción
id_transaccion (PK)	INTEGER	Clave primaria autoincremental
id_comprador_fk (FK)	INTEGER	FK → usuario.id_usuario (comprador)
id_vendedor_fk (FK)	INTEGER	FK → usuario.id_usuario (vendedor)
id_servicio_fk (FK)	INTEGER	FK → servicio.id_servicio
horas_transferidas	REAL	Cantidad de horas intercambiadas

estado	TEXT	Estado de la transacción (enum: PENDIENTE, ACEPTADA, COMPLETADA, CANCELADA)
timestamp	INTEGER	Timestamp de creación

VALORACION (tabla valoracion): representa la valoración que emite un vecino sobre otro tras una transacción completada.

Atributo	Tipo	Descripción
id_valoracion (PK)	INTEGER	Clave primaria autoincremental
id_transaccion_fk (FK)	INTEGER	FK → transaccion.id_transaccion
id_valorador_fk (FK)	INTEGER	FK → usuario.id_usuario (emisor)
id_valorado_fk (FK)	INTEGER	FK → usuario.id_usuario (receptor)
pictogramas_json	TEXT	Array de IDs de pictogramas ARASAAC serializado como JSON
comentario	TEXT (nullable)	Texto libre opcional
timestamp	INTEGER	Timestamp de la valoración

Relaciones y cardinalidades

Relación	Cardinalidad	Descripción
USUARIO publica SERVICIO	1 : N	Un usuario puede publicar múltiples servicios
USUARIO solicita TRANSACCION (como comprador)	1 : N	Un usuario puede realizar múltiples solicitudes
USUARIO ofrece TRANSACCION (como vendedor)	1 : N	Un usuario puede recibir múltiples solicitudes sobre sus servicios
SERVICIO genera TRANSACCION	1 : N	Un servicio puede tener varias transacciones a lo largo de su vida (por ejemplo, una cancelada y otra completada)
TRANSACCION recibe VALORACION	1 : N	Una transacción puede generar valoraciones asociadas (el diseño admite futuras valoraciones bidireccionales, aunque la lógica de negocio actual restringe el emisor al comprador)

USUARIO VALORACION valorador)	emite (como	1 : N	Un usuario puede emitir múltiples valoraciones
USUARIO VALORACION valorado)	recibe (como	1 : N	Un usuario puede recibir múltiples valoraciones

Multiplicidad de roles y normalización

Un aspecto destacable del diseño es que la entidad USUARIO participa en hasta cuatro relaciones diferentes apuntando a la misma tabla, cada una representando un rol distinto dentro de la lógica de negocio: comprador, vendedor, valorador y valorado. Podría haberse optado por tablas redundantes (por ejemplo, una tabla "comprador" y otra "vendedor"), pero esto habría roto la normalización y habría duplicado información innecesariamente. El enfoque adoptado mantiene una única tabla de usuarios y diferencia los roles mediante los nombres de las claves foráneas, lo que permite que un mismo usuario pueda ser comprador en una transacción y vendedor en otra, o valorador de un vecino y valorado por otro.

Decisiones de diseño complementarias

El campo *pictogramas_json* de la tabla VALORACION almacena el array de pictogramas seleccionados como una cadena JSON en lugar de modelarlo como una tabla N:M independiente. Esta desnormalización deliberada simplifica el esquema y es adecuada para el caso de uso: los pictogramas se consultan siempre en bloque junto con su valoración, nunca de forma independiente, y no se necesita buscar valoraciones por pictograma.

Todas las claves foráneas se definen con la política ON DELETE CASCADE: si un usuario es eliminado, todos sus servicios, transacciones y valoraciones asociadas también lo serán, evitando registros huérfanos en la base de datos.

[Diagrama Entidad-Relación de VecindApp. Elaboración propia en draw.io](#)

6.4. Diagrama de clases

El diagrama de clases refleja la estructura real del código tras los 6 sprints de desarrollo. A diferencia del diagrama inicial presentado en la fase de diseño (que contemplaba 13 clases en 5 capas), la arquitectura final se ha expandido a aproximadamente 45 clases agrupadas en cuatro capas según los principios de Clean Architecture. El diagrama completo, generado mediante la notación declarativa Mermaid, se incluye como Anexo X al ocupar un espacio considerable que haría poco legible su inclusión en el cuerpo de la memoria.

A continuación, se describen las capas y se comentan las clases más relevantes de cada una de ellas, así como las relaciones y decisiones de diseño que las articulan.

Capa Domain

Esta capa representa el núcleo del negocio y es totalmente independiente de Android, Room o cualquier framework externo. Se compone de dos paquetes:

- **domain.model** contiene los enums del dominio que modelan los conceptos del banco de tiempo vecinal: NivelVecino, CategoriaServicio, EstadoServicio, EstadoTransaccion y Barrio. Cada uno encapsula un conjunto cerrado de valores con significado de negocio (por ejemplo, el ciclo de vida de una transacción o los barrios de Zaragoza disponibles).
- **domain.repository** contiene las cuatro interfaces que definen los contratos de acceso a datos: UsuarioRepository, ServicioRepository, TransaccionRepository y ValoracionRepository. Estas interfaces no conocen Room ni SQLite, lo que permitiría en el futuro sustituir la fuente de datos (por ejemplo, añadir una API remota) sin modificar los ViewModels ni ningún otro componente de la capa superior.

Capa Data

Esta capa implementa la persistencia local ofreciendo al dominio lo que sus interfaces declaran. Se organiza en cuatro subpaquetes:

- **data.entities** contiene las cuatro entidades Room anotadas con *@Entity*: Usuario, Servicio, Transaccion y Valoracion. Cada una declara explícitamente sus claves primarias, índices y *foreign keys* con política ON DELETE CASCADE. Algunas entidades incluyen pequeñas funciones de negocio (por ejemplo, Usuario.calcularNivel() o Servicio.estaActivo()) que encapsulan lógica trivial cercana al estado del objeto.
- **data.db** contiene la clase abstracta AppDatabase (que extiende RoomDatabase con patrón Singleton), los cuatro DAOs como interfaces (UsuarioDao, ServicioDao, TransaccionDao, ValoracionDao), la clase Converters para transformar enums a String y viceversa, y la clase SeedDatabaseCallback que pobla la base de datos con datos iniciales en su primera creación.
- **data.repository** contiene las cuatro implementaciones de los contratos del dominio (UsuarioRepositoryImpl, ServicioRepositoryImpl, TransaccionRepositoryImpl y ValoracionRepositoryImpl). Cada implementación recibe su DAO por constructor y delega las operaciones simples, encapsulando la lógica atómica (como la transferencia de horas) cuando es necesaria.
- **data.SesionUsuario** es una clase helper que encapsula el acceso a SharedPreferences para gestionar el identificador del usuario con sesión activa, abstrayendo el resto de la aplicación de los detalles de almacenamiento clave-valor.

Capa Application

Esta capa está formada por una única clase, VecindAppApplication, que actúa como contenedor Singleton de dependencias. Instancia de forma perezosa (by lazy) la base de datos y los cuatro repositorios, exponiéndolos como propiedades públicas para que cualquier Fragment o Activity pueda acceder a ellos. Esta clase cumple el mismo propósito que Hilt o Dagger, pero con una implementación manual que evita introducir frameworks adicionales en el proyecto, como ya se justificó en el punto 4.

Capa Presentation

Esta es la capa más extensa del proyecto y la que más ha crecido respecto al diagrama inicial. Se organiza en múltiples subpaquetes correspondientes a las pantallas funcionales de la aplicación:

- **Raíz:** MainActivity (Single Activity que aloja el NavHostFragment y el BottomNavigationView) y MainViewModel (ViewModel transversal que expone el conteo reactivo de notificaciones mediante un StateFlow y la técnica flatMapLatest para reaccionar al cambio de usuario activo).
- **ui.auth:** LoginFragment + LoginViewModel y RegistroFragment + RegistroViewModel, responsables del flujo de autenticación local.
- **ui.escaparte:** EscaparteFragment, EscaparteViewModel y ServicioAdapter, que implementan la pantalla principal con el listado de servicios activos.
- **ui.servicio:** CrearServicioFragment, DetalleServicioFragment y sus respectivos ViewModels, que gestionan la creación, edición y consulta de servicios individuales.
- **ui.transaccion:** TransaccionFragment, TransaccionViewModel, TransaccionAdapter y el modelo de presentación TransaccionUI (que enriquece la entidad Transaccion con información específica de la UI como el rol del usuario y las acciones disponibles).
- **ui.perfil:** PerfilFragment + PerfilViewModel, que muestran el saldo, el nivel y los servicios publicados por el usuario.
- **ui.historial:** HistorialFragment, HistorialViewModel, HistorialAdapter y dos modelos de presentación (HistorialItem para las tarjetas de transacciones y DatoMensual para los puntos del gráfico MPAndroidChart).
- **ui.valoracion:** ValoracionBottomSheetFragment, DetalleValoracionBottomSheet, ValoracionViewModel y el helper PictogramaMapper, que implementan el sistema de valoración con pictogramas ARASAAC.
- **ui.common:** clases utilitarias transversales reutilizadas desde varios Fragments: TtsHelper (gestión del motor Text-to-Speech y funciones de formateo humanizado de

horas), CategoriaMapper (asignación dinámica de pictogramas por categoría) y SnackbarUtils (extensiones Kotlin para mostrar Snackbars uniformes).

Decisiones de diseño destacables

- Cada ViewModel incluye una clase anidada Factory que implementa ViewModelProvider.Factory, imprescindible cuando el ViewModel recibe dependencias por constructor. Estas factorías se han omitido del diagrama para evitar ruido visual, pero están presentes en el código y son las que permiten la inyección manual de los repositorios correspondientes.
- Se han introducido modelos de presentación específicos (TransaccionUI, HistorialItem, DatoMensual) que no se confunden con las entidades Room. Estos modelos enriquecen la información básica de la entidad con datos calculados o contextuales (rol del usuario, título del servicio asociado, flags de acciones permitidas) sin contaminar las entidades del dominio con lógica de presentación. Esta separación facilita que las entidades mantengan su responsabilidad única de modelar los datos persistidos.
- La comunicación entre ViewModels y repositorios se realiza siempre a través de las interfaces de domain.repository, nunca contra las implementaciones concretas. Esto materializa el principio de inversión de dependencias de Clean Architecture: las capas superiores dependen de abstracciones, no de detalles.
- Los repositorios no se comunican entre sí. Cuando un caso de uso requiere cruzar información de varias entidades (por ejemplo, completar una transacción necesita leer y modificar usuarios, servicios, transacciones y valoraciones), es el ViewModel correspondiente el que orquesta las llamadas por separado a cada repositorio. Esta regla se estableció durante el Sprint 1 tras detectar que el diseño inicial permitía llamadas cruzadas entre repositorios, lo que rompía el aislamiento de las fuentes de datos.
- Finalmente, todos los flujos de datos reactivos utilizan StateFlow y Flow de Kotlin Coroutines. Los ViewModels exponen sus estados como StateFlow inmutables y los Fragments se suscriben respetando el ciclo de vida mediante repeatOnLifecycle(Lifecycle.State.STARTED), garantizando que los recursos se liberen cuando la pantalla no está visible.

[Ver Anexo . Diagrama de Clases completo de VecindApp.](#)

7. Diseño del proyecto

Este apartado recoge las decisiones de diseño tomadas a lo largo del proyecto, desde la concepción visual inicial hasta la interfaz final implementada. Se describen el proceso creativo

previo, la paleta de colores y el sistema visual adoptados, las pantallas principales de la aplicación acompañadas de capturas reales, y las decisiones arquitectónicas más relevantes en materia de navegación y experiencia de usuario.

7.1. Concepción visual inicial

En la fase de arranque del proyecto se utilizó la herramienta generativa NanoBanana (integrada en Gemini) para obtener varios bocetos de inspiración sobre el estilo visual que podía adoptar la aplicación. Estos bocetos sirvieron únicamente como punto de partida para explorar diferentes estéticas y no como wireframes formales. La idea era definir un estilo moderno pero sencillo e intuitivo, apto para un público amplio que incluyera tanto usuarios jóvenes como personas mayores o con dificultades cognitivas.

Durante el desarrollo, el diseño evolucionó considerablemente respecto a aquellos bocetos iniciales conforme se iban incorporando las necesidades reales de cada pantalla. La decisión de partir de bocetos orientativos en lugar de wireframes formales permitió mantener la flexibilidad necesaria para ajustar la interfaz sprint a sprint, priorizando la funcionalidad sobre la fidelidad al diseño previo.

7.2. Paleta de colores y sistema visual

La paleta de colores se ideó en la fase del arranque del proyecto pero no se definió hasta el Sprint 6 tras detectar que el tema por defecto de Android Studio (morado y teal) no transmitía la identidad comunitaria y cercana que se buscaba para VecindApp. Se optó por una paleta verde y naranja sobre un tema Light:

- Verde VecindApp (color primario): transmite cercanía, naturaleza y comunidad. Se aplica a botones principales, barras de estado, Sliders y elementos activos de la interfaz.
- Naranja acento (color secundario): se usa para los FABs (Floating Action Buttons) y elementos que requieren llamar la atención del usuario, como la acción de publicar un nuevo servicio.
- Fondo gris verdoso claro: fondo general de la aplicación, suave a la vista y coherente con la paleta principal.
- Blanco arena sobre superficies: tarjetas Material (MaterialCardView) con fondo blanco que contrastan con el fondo general.

La configuración se realizó en el archivo themes.xml vinculando los colores de la paleta a los atributos del tema Material (colorPrimary, colorSecondary, colorBackground, colorSurface, etc.), lo que propagó automáticamente la paleta a todos los componentes Material de la aplicación (botones, FABs, Sliders, TabLayouts, etc.) sin necesidad de especificar colores hardcodeados en cada layout.

Se valoró implementar también un tema oscuro, pero se descartó para el MVP por no disponer de una paleta oscura suficientemente pulida. Dado que la arquitectura Material permite añadirlo a posteriori sin modificar las funcionalidades, se ha contemplado como una mejora para vías futuras.

7.3. Pantallas principales

A continuación, se describen las pantallas principales de la aplicación acompañadas de capturas reales tomadas del dispositivo físico.

Login y Registro



Las pantallas de autenticación siguen un flujo minimalista. El Login pide únicamente el nombre del vecino (sin contraseña, para facilitar la demo multiusuario) y ofrece un enlace directo al Registro. El Registro añade un Spinner con los barrios de Zaragoza predefinidos.

Figura 2. Captura de la pantalla de Login



Figura 3. Captura de la pantalla de Registro

Escaparate

Es la pantalla de inicio tras el login. Muestra un RecyclerView de tarjetas con todos los servicios **ACTIVOS** de la comunidad ordenados por fecha de publicación descendente. Cada tarjeta muestra el título, el coste en horas (con formato humanizado) y un pictograma **ARASAAC** asignado dinámicamente según la categoría del servicio. Un FAB con icono "+" permite publicar un nuevo servicio, y un segundo FAB con icono de altavoz activa el TTS para leer los servicios visibles en pantalla.

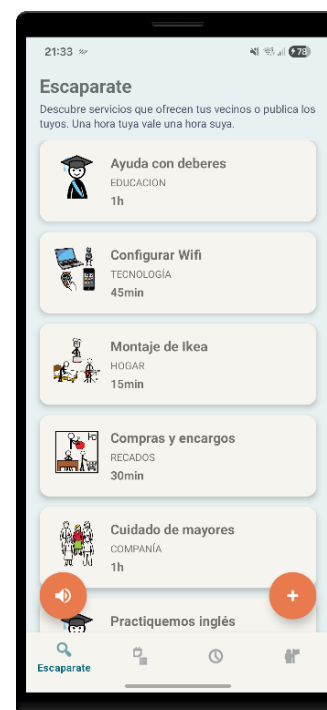


Figura 4. Captura del Escaparate con tarjetas de servicios y FABs.

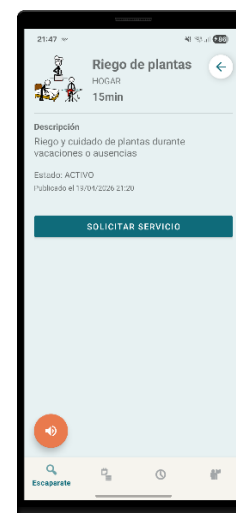
Crear y Detalle de servicio



Figura 5. Captura del formulario de creación de servicio con Slider

El formulario de creación de servicio se diseñó priorizando la prevención de errores de entrada. Destaca la decisión de sustituir el campo de texto libre para el coste en horas por un Slider Material con pasos de 0,25 h (es decir, cuartos de hora), con un rango de 0,25 a 8 horas. Esta decisión responde a que permitir valores arbitrarios como "1,37 horas" no tiene sentido en el contexto de un banco de tiempo vecinal. El Slider incluye un LabelFormatter que muestra el valor seleccionado con formato humanizado (por ejemplo, "1h 15min") mientras el usuario lo arrastra.

Figura 6 & 7. Capturas del Detalle del Servicio mostrando botones contextuales.



La pantalla de Detalle del Servicio es el hub central de navegación de la aplicación. Desde cualquier tarjeta del Escaparate, Perfil, Transacciones o Historial se accede a esta pantalla, que a su vez es el único punto desde el cual se puede abrir el BottomSheet modal de valoración. Los botones de acción mostrados cambian según el rol del usuario (propietario vs. no propietario) y el estado del servicio (ACTIVO, RESERVADO, COMPLETADO).

Transacciones

La pestaña de Transacciones muestra todas las transacciones activas del usuario, tanto aquellas en las que actúa como comprador como aquellas en las que actúa como vendedor. Se aplicó una diferenciación visual por rol mediante un chip coloreado en la parte superior de cada tarjeta ("Comprador" en azul, "Vendedor" en verde). Los botones de acción disponibles (Aceptar, Completar, Cancelar, Valorar) se calculan dinámicamente según el estado de la transacción y el rol del usuario.

Las tarjetas de transacciones canceladas reciben un tratamiento visual específico: se muestran oscurecidas, con el texto tachado mediante `Paint.STRIKE_THRU_TEXT_FLAG` y una leyenda "Sin cargo" que indica que no se ha producido transferencia de horas.

Un badge numérico rojo en la pestaña del BottomNav indica al usuario cuántas transacciones requieren su atención (pendientes de aceptar o completadas sin valorar).

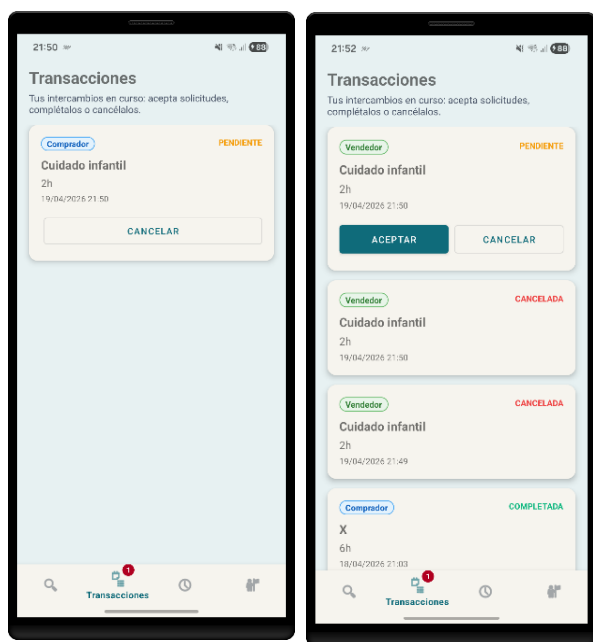


Figura 8.
Captura con una
tarjeta de
transacción
cancelada (texto
tachado y "Sin
carga")



Figuras 9 & 10. Capturas de las transacciones en función de rol.

Valoración con pictogramas ARASAAC

La valoración se realiza mediante un BottomSheet modal con 9 pictogramas ARASAAC seleccionables y un campo de texto libre opcional para comentarios. El BottomSheet es modal (no se apila en el back stack) por diseño: esto permite que al cerrarse el usuario vuelva exactamente a la pantalla desde la que lo invocó, evitando bucles de navegación.



La decisión de utilizar pictogramas en lugar de estrellas numéricas responde al objetivo de accesibilidad cognitiva: se eliminan las barreras que puedan tener usuarios con dificultades para interpretar valoraciones numéricas abstractas, sustituyéndolas por conceptos visuales concretos como "puntual", "amable" o "cumple lo prometido"

Figura 11. Captura del BottomSheet de valoración con los 9 pictogramas ARASAAC.

Perfil e Historial

El Perfil muestra la información del usuario (nombre, barrio, saldo de horas, nivel y número de intercambios completados) junto con la lista de servicios publicados por él. El nivel del vecino (Novato, Activo, Veterano, Referente) se recalcula automáticamente en función del número de intercambios y se muestra como una tarjeta destacada. El Historial cuenta con dos pestañas (completadas y canceladas) y un gráfico de barras agrupadas generado con MPAndroidChart que

visualiza el balance mensual de horas ganadas frente a horas gastadas.



Figura 13. Captura del Historial con el gráfico MPAndroidChart y las pestañas.

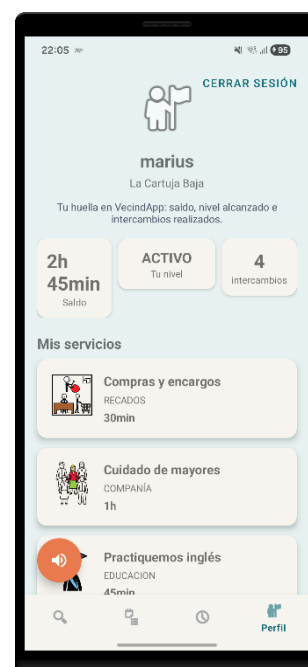


Figura 12. Captura del Perfil con las tarjetas de saldo, nivel e intercambios.

7.4. Decisiones de diseño destacables

- **Navegación centralizada en el Detalle del Servicio.** Se diseñó un modelo de navegación en forma de hub, donde cualquier tarjeta de servicio (sea del Escaparate, Perfil, Transacciones o Historial) lleva siempre al Detalle del Servicio. Desde ese Detalle se puede abrir el BottomSheet modal de valoración, pero no al revés. Esta decisión cortó de raíz los bucles de navegación circular que se habían detectado durante las pruebas funcionales del Sprint 4, cuando existía la posibilidad de navegar en cadena tarjeta → detalle → valoración → detalle → valoración indefinidamente.

- **Accesibilidad activa sobre pasiva.** El botón de TTS está visible como un elemento propio de la interfaz en todas las pantallas donde tiene sentido (Escaparate, Detalle, Perfil, Valoración). Se ha priorizado esta accesibilidad "activa" (el usuario pulsa un botón claramente visible) sobre la compatibilidad pasiva con lectores de pantalla del sistema como TalkBack, ya que no se puede asumir que un adulto mayor o una persona con dificultades cognitivas sepa configurar y activar TalkBack, pero sí puede pulsar un icono de altavoz.
- **Validación temprana en solicitudes de servicio.** Cuando un comprador solicita un servicio, el sistema valida previamente que su saldo de horas sea suficiente antes de crear la transacción y marcar el servicio como RESERVADO. Esta validación evita bloquear servicios en el escaparate por solicitudes que nunca podrán completarse, manteniendo así la oferta visible para otros vecinos con saldo suficiente.
- **Regla de valoración unidireccional.** Tras la refactorización del Sprint 6, solo el comprador puede valorar al vendedor. El vendedor completa la transacción y su ciclo termina ahí. Esta regla simplifica la lógica del badge de notificaciones (que ahora solo cuenta las valoraciones pendientes del comprador) y clarifica la experiencia de usuario evitando el automatismo del popup de valoración tras completar transacciones.
- **Feedback uniforme mediante Snackbars.** Se migró toda la aplicación de Toasts a Snackbars Material, aprovechando su comportamiento contextual (aparecen en la parte inferior de la pantalla, pueden incluir acciones) y su mejor integración visual con la paleta de la aplicación. Los Snackbars se posicionan correctamente incluso cuando aparecen encima de elementos como el botón de "Iniciar sesión" para no taparlos.

8. Despliegue y pruebas

Este apartado documenta el proceso de generación del ejecutable de la aplicación, las decisiones tomadas respecto a la distribución, la estrategia de pruebas aplicada a lo largo del desarrollo, y las evidencias recopiladas tanto de bugs detectados como de su resolución. Al tratarse de un desarrollo individual sin equipo de QA dedicado, la calidad del MVP final se ha apoyado en un proceso iterativo de pruebas funcionales integradas en cada sprint y documentadas en el diario de desarrollo del proyecto.

8.1. Despliegue de la aplicación

Al tratarse de una aplicación Android nativa offline-first, el despliegue se reduce a la generación de un fichero APK instalable en cualquier dispositivo Android 8.0 (API 26) o superior. No existe backend remoto, base de datos en la nube ni servicios externos que requieran configuración adicional: toda la persistencia se realiza en local mediante Room/SQLite sobre el propio dispositivo. Esto tiene dos consecuencias directas para el

despliegue: por un lado, simplifica enormemente la distribución (no hay servidores que mantener ni credenciales que gestionar); por otro, elimina los puntos de fallo externos habituales en aplicaciones conectadas (caídas de API, latencia de red, expiración de tokens).

Generación del APK

El APK se genera desde Android Studio mediante la tarea Gradle *assembleDebug*, que produce el fichero *app-debug.apk* en la carpeta *app/build/outputs/apk/debug/*. Esta es la versión que se entrega junto con la memoria, empaquetada en el ZIP del código fuente tal como exige la normativa del TFG.

```
./gradlew clean assembleDebug
```

El flag *clean* asegura que se parte de un estado limpio eliminando cualquier artefacto de compilaciones anteriores, evitando así el clásico problema de "funciona en mi máquina pero no al reinstalar". Tras un BUILD SUCCESSFUL, el APK queda listo para ser instalado.

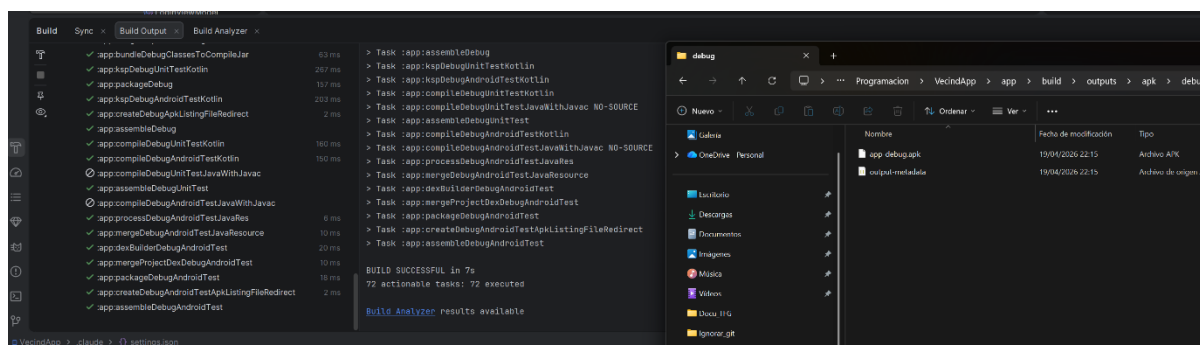


Figura 14. Captura de la salida de Gradle con el mensaje BUILD SUCCESSFUL tras generar el APK de release.

Instalación en dispositivo

Para la instalación se utiliza el comando:

```
./gradlew installDebug
```

Este comando requiere tener previamente habilitadas las Opciones de desarrollador y la Depuración USB en el dispositivo Android, así como aceptar la huella digital del ordenador la primera vez que se conecta. Alternativamente, el APK puede transferirse manualmente al dispositivo y abrirse desde cualquier explorador de archivos, lo que permite distribuirlo sin depender de Android Studio.

Entorno de verificación

Durante todo el desarrollo, la verificación se realizó en un dispositivo físico Android conectado al ordenador mediante cable USB, tal como se estableció en el objetivo general del proyecto. Esta decisión respondió a tres motivos concretos. En primer lugar, el motor TTS se comporta

de forma distinta en emulador y dispositivo real (el emulador puede no tener voces instaladas para el idioma solicitado, lo que obligaba a instalar paquetes adicionales). En segundo lugar, el rendimiento del RecyclerView, los BottomSheets y las animaciones Material solo reflejan su comportamiento real en hardware. Y, en tercer lugar, el tamaño y la densidad de la pantalla real permitieron detectar problemas de layout (truncado de texto en las tarjetas de nivel, posicionamiento de Snackbars) que no se apreciaban en el emulador.

Además, se utilizaron dos entornos de desarrollo distintos: un PC personal y un PC del trabajo. Este segundo entorno introdujo una complicación significativa por la presencia del firewall corporativo Zscaler, que interceptaba las conexiones HTTPS de Gradle e impedía la descarga de dependencias. La solución requirió extraer el certificado de Zscaler en formato .der, importarlo al almacén de certificados Java con keytool y configurar gradle.properties globalmente. Tras documentar este proceso en un manual interno (manual_gradle_zscaler.md), la sincronización entre ambos entornos dejó de ser un problema.

8.2. Estrategia de pruebas

La normativa del CFGS indica que las pruebas se realicen mediante caja negra sobre los casos de uso definidos. Este enfoque verifica que cada funcionalidad se comporte según lo especificado, sin importar los detalles internos de la implementación: se proporcionan unos datos de entrada y se comprueba que la salida o el efecto observable coincide con lo esperado.

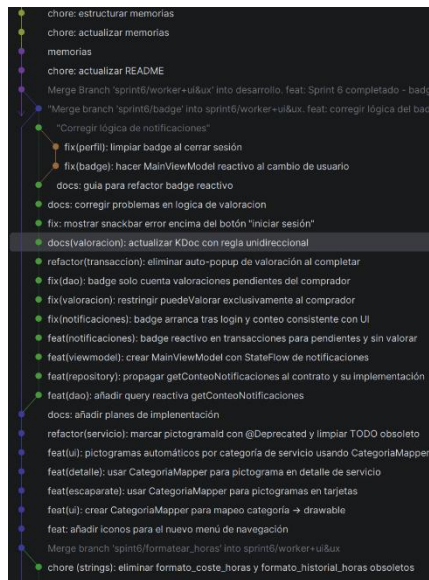
Integración continua del testing en el desarrollo

Para este proyecto, las pruebas no se han tratado como una fase posterior aislada, sino como una actividad continua integrada en el ciclo de vida de cada sprint. Al finalizar cada sprint se ejecutaban las pruebas funcionales correspondientes al caso de uso implementado y, solo cuando estas pasaban sin errores, se procedía al merge de la rama a desarrollo y se abría el siguiente sprint. Este enfoque respondía a una filosofía clara: atacar los errores mientras el código está fresco en la memoria. Introducir una pila de bugs acumulados del Sprint 2 durante el Sprint 5 habría sido un infierno, porque para entonces la arquitectura de la UI y los flujos de datos ya habrían evolucionado significativamente.

Esta filosofía se materializa en las memorias de desarrollo del proyecto, donde cada jornada de trabajo incluye una sección específica de pruebas funcionales con el formato "Problema → Solución", que posteriormente ha alimentado la lista de bugs documentados en este apartado.

Testing manual vs. automatizado

Se optó por pruebas manuales en lugar de tests automatizados (JUnit/Espresso) por tres motivos. Primero, la normativa del CFGS pide explícitamente pruebas de caja negra sobre casos de uso, no tests unitarios. Segundo, muchas de las funcionalidades críticas de VecindApp dependen de interacciones táctiles complejas (Slider con pasos de 0,25 h,



BottomSheets modales, TTS en tiempo real, navegación entre Fragments) que son difíciles y costosas de automatizar correctamente con Espresso. Y tercero, el tiempo invertido en configurar una suite de tests automatizados habría causado retraso en el desarrollo de funcionalidades del MVP, lo cual era la prioridad para un desarrollo individual con plazo ajustado.

Como contraparte de esta decisión, se ha compensado con una documentación exhaustiva del proceso de testing manual, trazable a través del historial de Git mediante commits del tipo *fix*: y de las entradas correspondientes en las memorias del proyecto.

Figura 15. Captura de árbol en Git.

Enfoque iterativo: el ejemplo de la valoración unidireccional

Un ejemplo claro de cómo las pruebas funcionales no solo detectan bugs de código sino también mejoras de lógica de negocio se produjo en el Sprint 6 con la regla de valoración. El código funcionaba correctamente: ambos usuarios (comprador y vendedor) podían valorarse mutuamente tras completar la transacción. Sin embargo, al ejecutar la prueba funcional completa, se detectaron dos problemas simultáneos: el badge de notificaciones contaba incorrectamente las transacciones sin valorar para el vendedor (que según la lógica de negocio no debería valorar), y el auto-popup de valoración se abría al vendedor al completar la transacción, lo que era incoherente.

La solución no fue un *fix* de código concreto, sino una refactorización de la lógica de negocio: se estableció la regla unidireccional de que solo el comprador valora al vendedor, se modificó la query del DAO (`getConteoNotificaciones`) para filtrar por `id_comprador_fk`, se eliminó el auto-popup, y se actualizó la documentación KDoc. Este tipo de hallazgos justifica por sí solo el enfoque iterativo de testing aplicado al proyecto.

8.3. Pruebas funcionales sobre los casos de uso

Se ejecutó una batería de pruebas de caja negra sobre cada uno de los 12 casos de uso definidos en el apartado 6.2. Para cada caso se verificaron el flujo principal, los flujos alternativos y las excepciones documentadas en su especificación. La tabla siguiente resume el resultado final tras las iteraciones de corrección:

CU	Caso de uso	Flujo principal	Flujos alternativos / Excepciones	Resultado
CU-01	Registrarse como nuevo vecino	OK	Nombre duplicado rechazado con Snackbar	OK
CU-02	Iniciar sesión	OK	Usuario inexistente rechazado	OK
CU-03	Cerrar sesión	OK	Badge se limpia antes de navegar al Login	OK
CU-04	Explorar escaparate	OK	FAB de TTS lee servicios visibles	OK
CU-05	Escuchar contenido (TTS)	OK	Se detiene al cambiar de pantalla	OK
CU-06	Publicar servicio	OK	Título vacío rechazado; Slider funcional en pasos de 0,25 h	OK
CU-07	Editar o eliminar servicio	OK	Botones ocultos si servicio no está ACTIVO	OK
CU-08	Ver detalle del servicio	OK	Botones contextuales según rol y estado	OK
CU-09	Solicitar servicio	OK	Saldo insuficiente rechazado; propio servicio rechazado	OK
CU-10	Completar transacción	OK	Operación atómica verificada en pruebas concurrentes	OK
CU-11	Valorar con pictogramas ARASAAC	OK	Regla unidireccional; no duplicados; mínimo un pictograma obligatorio	OK
CU-12	Consultar perfil e historial	OK	Gráfico MPAndroidChart; pestañas completadas/canceladas	OK

Tabla 8. Resultado de las pruebas funcionales sobre los casos de uso. Elaboración propia.

8.4. Bugs detectados y resueltos

Durante las pruebas funcionales se detectaron y resolvieron múltiples bugs a lo largo de los sprints. La siguiente tabla recoge los más representativos por su impacto en la experiencia de usuario o en la integridad de los datos, clasificados por severidad:

ID	Sprint	Severidad	Síntoma detectado	Solución aplicada
BUG-01	S2	Crítica	Foreign Key constraint failed al crear el primer servicio	SeedDatabaseCallback crea usuarios iniciales al arrancar la BBDD
BUG-02	S2	Alta	RecyclerView del Escaparate no renderizaba tarjetas	Mover la asignación de layoutManager y Adapter fuera del lambda del onClick
BUG-03	S2	Media	Crash en arranque: "You need to use a Theme.AppCompat theme"	Cambiar el parent del tema a Theme.MaterialComponents.Light.NoActionBar
BUG-04	S3	Crítica	Sobreescritura del saldo al completar transacción	Incluir el cambio de saldo en el .copy() del update() completo
BUG-05	S3	Alta	No se validaba el saldo antes de solicitar un servicio	Injectar UsuarioRepository en DetalleServicioViewModel
BUG-06	S3	Media	Tarjetas COMPLETADAS sin botón de valorar	Añadir campo puedeValorar a TransaccionUI y botón en el adapter
BUG-07	S3	Baja	Historial no mostraba título del servicio	Crear HistorialItem enriquecido con el título del servicio asociado
BUG-08	S3	Baja	Transacciones canceladas no aparecían en historial	Añadir segundo Flow para CANCELADAS y sección diferenciada
BUG-09	S3	Baja	Tarjeta de nivel no se adaptaba a textos largos	Cambiar wrap_content por match_parent con autoSizeMinTextSize

BUG-10	S4	Alta	Usuarios duplicados en la BBDD	Añadir índice UNIQUE en la columna nombre
BUG-11	S4	Media	BottomSheet mostraba siempre "Valoración recibida"	Pasar booleano esEnviada por argumentos
BUG-12	S5	Media	TTS leía mal palabras en mayúsculas sin tildes	Corregir enums añadiendo tildes (TECNOLOGÍA, COMPAÑÍA)
BUG-13	S5	Baja	TTS del detalle leía pero no indicaba a qué hacía referencia	Refactorizar DetalleServicioFragment.kt para anteponer etiquetas ("categoría", "coste", "descripción")
BUG-14	S5	Media	BottomSheet no actualizaba la lista al cerrarse	Implementar onDismissCallback para forzar recarga en el ViewModel padre
BUG-15	S6	Alta	Badge "pegado" al cerrar sesión	setUsuariold(SIN_SESION) antes de navegar al Login
BUG-16	S6	Crítica	Badge no reactivo al cambiar de usuario	Refactor del MainViewModel con MutableStateFlow + flatMapLatest
BUG-17	S6	Media	Auto-popup de valoración al completar transacción	Eliminar auto-popup; el comprador pulsa "Valorar" cuando quiera
BUG-18	S6	Baja	Snackbar de error tapado por teclado	Ajustar la posición para aparecer por encima del botón "Iniciar sesión"
BUG-19	S6	Media	Servicio editable en cualquier estado	Condicionar visibilidad de Editar/Eliminar a esPropietario && estado == ACTIVO

Tabla 9. Selección de bugs detectados y resueltos durante las pruebas funcionales.

Elaboración propia.

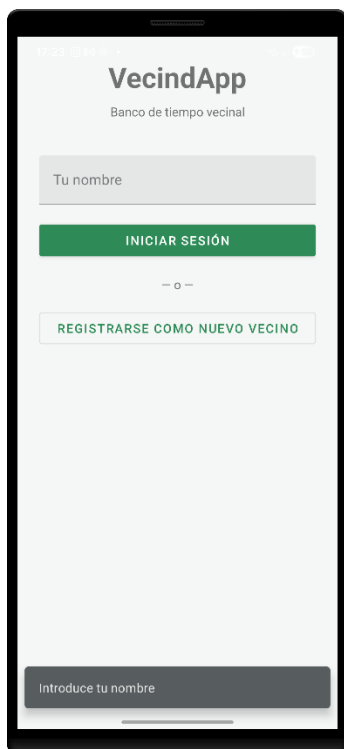


Figura 16. Captura del Snackbar de error en el login antes del fix. Quedaba tapado por el teclado.



Figura 17. Captura del mismo Snackbar tras el fix, posicionado correctamente encima del botón.

Clasificación por severidad

Para la tabla anterior se ha aplicado la siguiente escala:

- **Crítica:** el bug impide el funcionamiento del caso de uso o compromete la integridad de los datos (ej. crash, corrupción de saldo).
- **Alta:** el bug degrada significativamente la experiencia de usuario o la lógica de negocio, pero no impide el uso de la app.
- **Media:** el bug causa un comportamiento incorrecto o confuso, pero existe una forma alternativa de usar la funcionalidad.
- **Baja:** el bug es estético, de usabilidad secundaria o de baja frecuencia.

Del análisis de la tabla se desprende que los Sprints 3 y 6 concentraron el mayor número de bugs detectados. En el Sprint 3 esto se debió a la complejidad del ciclo de vida de las transacciones (la lógica más crítica del proyecto), y en el Sprint 6 a la integración tardía del sistema de notificaciones reactivas, que requirió varios ajustes finos sobre el MainViewModel y el flujo de sesión.

8.5. Prueba de humo funcional

Como verificación final del MVP se ejecutó una prueba de humo completa (también conocida como "smoke test" o prueba de fuego) simulando el ciclo de vida completo de una transacción

entre dos usuarios reales en el dispositivo. Esta prueba integra varios casos de uso encadenados y representa el flujo más común de la aplicación:

1. Usuario A (Amanda) inicia sesión y publica un servicio ("Ayuda con el ordenador", 1 h, categoría Tecnología).
1. Se cierra sesión. Usuario B (Marius) inicia sesión.
2. Usuario B explora el escaparate y comprueba que el servicio aparece en primera posición.
3. Usuario B pulsa el servicio para ver el detalle, activa el TTS y escucha la lectura completa.
4. Usuario B solicita el servicio. Se comprueba que su saldo (5 h iniciales) es suficiente y la transacción queda en estado PENDIENTE.
5. Se cierra sesión. Usuario A inicia sesión y ve la transacción en la pestaña Transacciones con un badge de notificación.
6. Usuario A acepta la solicitud (estado ACEPTADA) y posteriormente la completa.
7. Se verifica que el saldo de A ha aumentado en 1 h y el de B ha disminuido en 1 h.
8. Se verifica que el servicio ha cambiado a estado COMPLETADO y ya no aparece en el escaparate.
9. Se cierra sesión. Usuario B inicia sesión y ve el badge numérico indicando que tiene una transacción pendiente de valorar.
10. Usuario B pulsa "Valorar", selecciona los pictogramas "puntual" y "muy recomendable", añade un comentario, y envía la valoración.
11. Se verifica que el historial de ambos refleja la transacción completada con el signo adecuado (+1 h para A, -1 h para B), y que el gráfico MPAndroidChart se ha actualizado con los nuevos datos.
12. Usuario B consulta la valoración enviada desde el historial; Usuario A consulta la valoración recibida desde su perfil.
2. Se cierra sesión. Usuario B inicia sesión.
3. Usuario B explora el escaparate, pulsa el servicio, lo escucha con TTS y lo solicita.
4. Se cierra sesión. Usuario A inicia sesión y acepta la solicitud pendiente.
5. Usuario A completa la transacción. Se verifica que su saldo ha aumentado en 1 h.
6. Se cierra sesión. Usuario B inicia sesión y recibe el badge numérico en el BottomNav.
7. Usuario B pulsa "Valorar", selecciona los pictogramas "puntual" y "muy recomendable", añade un comentario, y envía la valoración.
8. Se verifica que el historial de ambos refleja la transacción completada con el signo + o - adecuado, y que el gráfico MPAndroidChart se actualiza con los nuevos datos.

8.6. Limitaciones del proceso de pruebas

Se considera necesario reconocer las limitaciones del enfoque de testing aplicado para que el tribunal disponga de una visión honesta del estado del proyecto:

- **Ausencia de tests automatizados:** no se dispone de una batería de tests unitarios (JUnit) ni de tests de interfaz (Espresso), lo que significa que las regresiones no se detectan automáticamente al introducir cambios. Cualquier modificación futura requeriría volver a ejecutar manualmente las pruebas funcionales.
- **Pruebas en un único dispositivo físico:** las pruebas se ejecutaron en un único modelo de dispositivo. No se ha validado el comportamiento en dispositivos con pantallas muy pequeñas (por debajo de 5 pulgadas), con resoluciones muy altas (tablets) ni con versiones de Android anteriores a la API 26.
- **Sin pruebas de rendimiento o carga:** al ser una aplicación offline sin concurrencia multiusuario real, no se han realizado pruebas de rendimiento con bases de datos grandes (miles de registros). La operación atómica de `@Transaction` de Room se ha verificado en escenarios de uso normal, pero no bajo estrés.
- **Sin pruebas de seguridad formales:** al no haber comunicación con servidores remotos, no se han realizado pruebas de seguridad (SQL injection, XSS, autenticación). La ausencia de un sistema de login con contraseña real ya está contemplada como debilidad en el DAFO.

9. Conclusiones

Este apartado recoge las conclusiones finales del proyecto, contrastando los objetivos alcanzados con los inicialmente propuestos, explicando las causas de aquellos que no se han podido completar, y compartiendo una reflexión personal sobre la experiencia vivida durante el desarrollo.

9.1. Objetivos alcanzados

De los ocho objetivos específicos definidos en el apartado 1.3, los ocho se han cumplido en la versión final entregada. La aplicación implementa las cuatro entidades Room con sus relaciones de clave foránea, el sistema de transacciones atómicas para el intercambio de horas, el TTS nativo con botón visible en las pantallas principales, el sistema de valoraciones con pictogramas ARASAAC bajo la regla unidireccional del comprador, el gráfico de barras agrupadas con `MPAndroidChart`, la autenticación local con nombres únicos, la categorización visual mediante pictogramas dinámicos y el sistema reactivo de notificaciones con badge en el `BottomNavigationView`. Además, la arquitectura MVVM + Clean Architecture ha escalado desde las 13 clases iniciales del diseño previo hasta aproximadamente 45 clases organizadas

en cuatro capas, manteniendo la separación de responsabilidades sin degradación apreciable del rendimiento.

Cabe destacar que varios de estos objetivos se superaron en alcance respecto a lo planificado inicialmente. El TTS, que en la propuesta original se limitaba a leer título y descripción de los servicios, acabó integrándose en las cuatro pantallas principales con formateo humanizado de horas y adaptación contextual (por ejemplo, distinguiendo entre "valoración enviada" y "valoración recibida"). El sistema de valoraciones, que inicialmente contemplaba valoraciones bidireccionales automáticas al completar la transacción, evolucionó durante las pruebas funcionales hacia un modelo unidireccional y voluntario más coherente con la lógica real de un banco de tiempo (inspirado en plataformas como BlaBlaCar). Y el sistema de notificaciones, originalmente pensado como tarjetas estáticas, terminó implementándose como un StateFlow reactivo capaz de actualizarse automáticamente al cambiar de usuario mediante flatMapLatest, un patrón que al inicio del proyecto resultaba completamente desconocido.

9.2. Objetivos no alcanzados

El principal objetivo que se descartó del alcance final fue la implementación del WorkManager para notificaciones locales programadas (caducidad de servicios, recordatorios). Se contemplaba en el diagrama de clases inicial (con una clase CaducidadWorker en su propia capa worker) y formaba parte del backlog del Sprint 6, pero se descartó por limitaciones de tiempo. La decisión se tomó de forma consciente en el Sprint 6, priorizando el sistema de notificaciones reactivas mediante badge (que aportaba más valor inmediato a la experiencia de usuario) frente a la programación de tareas en background. Este objetivo se traslada íntegramente a la sección de Vías Futuras.

Un segundo objetivo parcialmente no alcanzado es el de la implementación de Firebase Firestore para la recolección anónima de métricas de uso. Aunque estaba justificado en la propuesta inicial, se descartó del MVP al reconocerse que su implementación correcta requería un backend remoto, un sistema de identificación anónima de dispositivos y una política de privacidad, elementos incompatibles con la filosofía offline-first adoptada para el MVP. Firebase se mantendrá como una línea de mejora opcional en Vías Futuras.

Finalmente, ciertas ideas de refinamiento visual que surgieron durante las pruebas funcionales (modo oscuro, pictogramas personalizables, filtros avanzados por categoría en el Escaparate) se apuntaron como mejoras deseables pero no se abordaron para no comprometer la estabilidad del MVP. La decisión repetida a lo largo del proyecto fue clara: terminar primero, optimizar después, una regla personal que quedó reflejada incluso en un archivo específico del repositorio (cosas_para_aprender_luego.md) para evitar caer en rabbit holes.

9.3. Experiencia vivida y dificultades

Enfrentarme a este TFG compaginando una jornada laboral completa ha sido, probablemente, la mayor dificultad no técnica del proyecto. La planificación inicial con sprints de una semana se vio en la práctica distorsionada por la disponibilidad real de tiempo: había jornadas en las que apenas se podía dedicar una hora al desarrollo, seguidas de fines de semana con sesiones largas en las que se avanzaba más de lo previsto. El uso intensivo del diario de desarrollo (memorias.txt) ha sido fundamental para no perder el hilo entre sesiones y para poder retomar exactamente donde se había dejado el trabajo, algo que considero una de las mejores decisiones metodológicas del proyecto.

En el terreno técnico, algunas de las dificultades más destacables han sido las siguientes. La migración de KAPT a KSP en el Sprint 1, realizada al detectar que el catálogo de versiones de Gradle no incluía la versión de KAPT y que KAPT ya estaba deprecado. Aunque inicialmente era un problema a resolver, acabó siendo una oportunidad para adoptar la herramienta moderna y un primer contacto con la realidad de que en el desarrollo Android moderno "si algo está obsoleto, mejor actualizarse que parchear". Los certificados SSL de Zscaler en el ordenador del trabajo, que durante varios días impidieron la descarga de dependencias de Gradle y obligaron a aprender sobre keytool, trustStore de Java y configuración global de Gradle. Aunque costosa en tiempo, esta dificultad enseñó aspectos reales del desarrollo en entornos corporativos que normalmente no se abordan en un ciclo formativo. Y la gestión de estados reactivos en el sistema de notificaciones del Sprint 6, que requirió entender a fondo conceptos como StateFlow, flatMapLatest y el problema del "state bleed" en arquitecturas Single-Activity, conceptos que al inicio del proyecto resultaban completamente abstractos y que hoy forman parte de la caja de herramientas habitual.

---Otro aspecto relevante ha sido el trabajo con modelos de lenguaje e IA generativa. Durante el proyecto se han utilizado varias herramientas de IA como apoyo (principalmente Claude y Gemini), lo que ha permitido mantener un ritmo de desarrollo alto pero ha introducido desafíos nuevos: aprender a delimitar tareas para que no se pierdan en contexto, contrastar el código sugerido línea a línea antes de integrarlo, reconocer que una IA puede escribir código que compila pero que no tiene sentido para el usuario final (por ejemplo, un icono de silencio en un botón que activa la voz), y gestionar el historial de Git para que los commits generados por IA no ensuciasen el historial académico. Herramientas como git cherry-pick y git rebase --interactive han pasado de ser conceptos desconocidos a ser parte del flujo de trabajo diario, y la adopción del estándar Conventional Commits ha transformado el historial de Git en una documentación viva del proyecto.

En lo puramente personal, este proyecto ha supuesto el primer trabajo técnico de envergadura que he afrontado. Al principio costaba saber por dónde empezar ("no sabía si

ponerme con el diseño de la base de datos o con las interfaces en XML"), y el proceso de redactar la memoria ha sido prácticamente tan complejo como el propio desarrollo de la aplicación. Sin embargo, el proceso de escritura ha servido para organizar el conocimiento adquirido y para cerrar cada capítulo con la sensación de que efectivamente se ha aprendido algo concreto. Algunas decisiones técnicas que al comienzo parecían obvias (como permitir que los repositorios se llamasen entre sí) se revelaron incorrectas al estudiar a fondo los principios de Clean Architecture, y otras que parecían ortodoxas (como aplicar Scrum al 100% en un desarrollo individual) se relajaron para encajar con la realidad de un equipo unipersonal. En conjunto, el balance es muy positivo. El MVP entregado cumple con todos los objetivos específicos propuestos y va más allá en varios de ellos; la arquitectura del código permite incorporar las funcionalidades de Vías Futuras sin necesidad de refactorizaciones profundas; y la experiencia de haber abordado un proyecto completo desde el diseño inicial hasta el despliegue final deja un aprendizaje que excede con mucho lo adquirido en las asignaturas individuales del CFGS.

10. Vías futuras

Aunque el MVP entregado cumple con los objetivos específicos propuestos, a lo largo del desarrollo han surgido numerosas ideas, funcionalidades descartadas por tiempo y mejoras técnicas que quedan documentadas como líneas de evolución del proyecto. Este apartado las organiza en bloques temáticos atendiendo a su naturaleza: funcionalidades pendientes de la planificación inicial, mejoras técnicas, ampliaciones de accesibilidad y funcionalidades nuevas surgidas durante el desarrollo.

10.1. Funcionalidades descartadas del MVP

- **WorkManager para notificaciones locales programadas.** Esta fue la principal funcionalidad planificada en el Sprint 6 que se descartó por falta de tiempo. La idea original contemplaba una clase `CaducidadWorker` encargada de notificar al usuario cuando alguno de sus servicios publicados estuviera próximo a caducar, recordarle las valoraciones pendientes tras un periodo de inactividad, o avisar de nuevas transacciones sin necesidad de tener la aplicación abierta. La arquitectura actual ya contempla la entidad `Servicio` con un campo `fecha_caducidad` (actualmente opcional y no utilizado), por lo que la integración futura del `WorkManager` requeriría principalmente implementar la lógica del `Worker` y los canales de notificación del sistema Android.
- **Firebase Firestore para métricas anónimas.** Contemplada en la propuesta inicial, se descartó del MVP al reconocerse que una implementación correcta exigía backend remoto, sistema de identificación anónima y política de privacidad, elementos

incompatibles con la filosofía offline-first. Si el proyecto evolucionara hacia una fase comunitaria con múltiples usuarios reales, Firebase Firestore o una solución similar permitiría recoger métricas agregadas (número de usuarios activos, horas intercambiadas por barrio, distribución de categorías) de utilidad para medir el impacto social de la aplicación.

10.2. Mejoras técnicas y de arquitectura

- **Migración a Hilt para la inyección de dependencias.** El MVP utiliza inyección manual a través de VecindAppApplication por las razones justificadas en el apartado 4. No obstante, si el proyecto creciese hasta superar los 10-15 repositorios o se organizase en múltiples módulos Gradle, la migración a Hilt simplificaría la gestión de dependencias y reduciría el código repetitivo de las factorías de los ViewModels. La arquitectura actual facilita esta migración porque los repositorios ya están definidos como interfaces en la capa de dominio.
- **Backend remoto y sincronización con Retrofit.** El diseño offline-first ha sido una fortaleza del MVP, pero también es su mayor limitación: un usuario no puede usar la aplicación desde dos dispositivos distintos. Añadir un backend remoto (Firebase, una API REST propia con Retrofit, o una solución como Supabase) permitiría que cada usuario accediese a sus datos desde cualquier dispositivo y que los servicios publicados fuesen visibles por toda una comunidad real. La arquitectura Clean actual facilitaría esta integración, ya que bastaría con implementar una nueva capa de datos remota sin tocar los ViewModels ni la lógica de dominio.
- **Suite de tests automatizados.** Como ya se reconoció en el apartado 8, el proyecto no dispone de tests automatizados (JUnit/Espresso). Añadir una batería de tests unitarios sobre los repositorios y los ViewModels, junto con tests de interfaz con Espresso sobre los flujos principales, permitiría detectar regresiones de forma automática y facilitaría la incorporación de nuevas funcionalidades sin miedo a romper lo existente.
- **Sistema de autenticación con contraseña real.** El login por selección de perfil sin contraseña fue una decisión consciente para facilitar la demo multiusuario, pero no es viable en un entorno de producción. La evolución natural sería implementar autenticación con contraseña hasheada (bcrypt o similar), recuperación de contraseña por correo y, opcionalmente, autenticación biométrica (huella dactilar o reconocimiento facial) aprovechando la API BiometricPrompt de Android.
- **Migraciones de Room manuales.** Durante el desarrollo se ha utilizado fallbackToDestructiveMigration, que borra y recrea la base de datos al detectar un cambio de versión. En producción esto sería catastrófico (se perderían los datos de

todos los usuarios al publicar una actualización con cambios de esquema). Implementar migraciones manuales con `addMigrations(...)` y definir cada paso de evolución del esquema es un requisito imprescindible para cualquier despliegue real.

10.3. Mejoras de accesibilidad

- **Tema oscuro.** En el Sprint 6 se valoró implementar un tema oscuro, pero se descartó por no disponer de una paleta oscura suficientemente pulida. La arquitectura Material Components lo facilita: bastaría con crear `res/values-night/themes.xml` con la variante oscura de los colores y permitir al usuario alternar entre ambos desde los ajustes. La experiencia para usuarios con sensibilidad a la luz o que usen la aplicación de noche mejoraría notablemente.
- **Integración con TalkBack y navegación completa por teclado.** El TTS actual cubre la accesibilidad activa, pero no sustituye a TalkBack para usuarios con ceguera total. Completar el etiquetado de todos los componentes con `contentDescription`, revisar el orden de foco y probar la aplicación con TalkBack activo permitiría cumplir con las pautas WCAG 2.2 AA al 100% y abrir la aplicación a un público adicional.
- **Personalización del TTS.** Actualmente el TTS usa la voz y velocidad por defecto del sistema. Permitir al usuario elegir entre varias voces instaladas, ajustar la velocidad de lectura e incluso la tonalidad desde los ajustes mejoraría la experiencia de quienes dependen del TTS como canal principal.
- **Ampliación del catálogo de pictogramas ARASAAC.** La versión actual utiliza 9 pictogramas para valoraciones y 6 para categorías. Ampliar este catálogo permitiría valoraciones más expresivas (por ejemplo, pictogramas negativos como "no cumplió lo acordado") y una categorización más rica de los servicios.

10.4. Funcionalidades nuevas surgidas durante el desarrollo

- **Filtros y búsqueda avanzada en el escaparate.** El MVP muestra todos los servicios activos ordenados por fecha de publicación descendente. Durante las pruebas funcionales surgieron varias ideas de mejora: filtrar por categoría, por rango de coste, por distancia al barrio del usuario, o por vecino valorador (servicios de vecinos con valoración alta). El `ServicioDao` ya dispone del método `getByCategoria()` que no se llegó a conectar a la UI, por lo que implementar al menos el filtro por categoría sería una primera iteración de bajo coste.
- **Mensajería directa entre vecinos.** Una de las carencias evidentes del MVP es que comprador y vendedor no pueden comunicarse dentro de la app antes de la transacción para acordar detalles (hora, lugar, material necesario). Añadir un sistema simple de mensajería por transacción mejoraría significativamente la experiencia,

aunque requeriría el backend remoto mencionado anteriormente para que funcione correctamente entre dispositivos.

- **Notificaciones push.** Complementarias al WorkManager local. Si se incorpora un backend remoto, las notificaciones push (vía Firebase Cloud Messaging) permitirían avisar al vendedor cuando llegue una solicitud, al comprador cuando se acepte o complete, o a ambos cuando llegue una valoración.
- **Gamificación avanzada.** El sistema de niveles de vecino (Novato, Activo, Veterano, Referente) es una primera aproximación muy sencilla a la gamificación. Ampliarlo con insignias por tipo de servicio ofrecido (por ejemplo, "Experto en tecnología" tras 5 servicios en esa categoría), un ranking mensual de vecinos más activos o misiones semanales podría fomentar la participación y la creación de hábito.
- **Personalización del perfil.** Actualmente el avatar del usuario se define como String? avatarPath pero no se utiliza en la interfaz (no se ha integrado Glide). Permitir al usuario subir una foto de perfil desde la galería o la cámara, o elegir entre un conjunto de avatares predefinidos, daría mayor identidad a cada vecino dentro de la comunidad.
- **Mejoras en el gráfico MPAndroidChart.** El gráfico actual muestra barras agrupadas con las horas ganadas y gastadas por mes. Durante las pruebas funcionales se identificó que el gráfico, aunque funcional, no es del todo intuitivo (permite zoom in pero no zoom out, el desplazamiento recuerda a Google Maps). Mejoras posibles incluyen: filtros temporales (últimos 7 días, mes actual, año), selección de métricas distintas (número de transacciones vs. horas), y exportación del gráfico como imagen para compartir.

10.5. Evolución del alcance del proyecto

- **Distribución en Google Play Store.** Como se indicó en el apartado 8, la aplicación se entrega como APK de instalación directa. Publicarla en Google Play (tras adquirir la licencia de desarrollador, preparar las capturas para la ficha de la tienda y pasar la revisión de Google) permitiría llegar a usuarios reales más fácilmente y recibir feedback orgánico para iteraciones futuras.
- **Internacionalización (i18n).** Todos los textos visibles están ya extraídos a res/values/strings.xml, por lo que la base para traducir la aplicación a otros idiomas está puesta. El siguiente paso sería crear los archivos res/values-en/strings.xml, res/values-ca/strings.xml u otros, y configurar también voces TTS para cada idioma. Esto permitiría extender la aplicación a comunidades vecinales más allá de Zaragoza.
- **Ampliación geográfica.** El enum Barrio contiene actualmente los 36 barrios de Zaragoza hardcodedos. Sustituir este enum por una tabla Barrio en la base de datos,

alimentada dinámicamente (por ejemplo, desde un servicio de geolocalización o un conjunto de datos abierto), permitiría desplegar la aplicación en cualquier ciudad española.

- **Pilotaje con usuarios reales.** Antes de cualquier despliegue comercial, un pilotaje controlado con un pequeño grupo de vecinos reales (10-20 personas) de un barrio concreto permitiría validar las hipótesis del proyecto: ¿se usa la app para intercambiar tiempo?, ¿el sistema de valoración con pictogramas es efectivamente accesible?, ¿el saldo inicial de 5 horas es suficiente para generar actividad? Los resultados de este pilotaje guiarían las siguientes iteraciones.

•

11. Bibliografía / Webgrafía

Las referencias se han organizado en secciones temáticas que agrupan las fuentes por naturaleza (estudios, legislación, plataformas, documentación técnica, etc.), siguiendo las normas de publicación de la American Psychological Association (APA, 7ª edición).

Estadísticas demográficas y socioeconómicas

Instituto Nacional de Estadística. (2024). *Encuesta de Condiciones de Vida (ECV). Año 2024. Resultados definitivos*. <https://ine.es/dyngs/Prensa/es/ECV2024.htm>

Red Europea de Lucha contra la Pobreza y la Exclusión Social en el Estado Español. (2023). *Informe AROPE 2023: El Estado de la Pobreza. Seguimiento del indicador de pobreza y exclusión social en España 2008-2022*. EAPN España. https://www.eapn.es/ARCHIVO/documentos/documentos/1698046532_informe_aro-pe-2023_completo.pdf

European Anti-Poverty Network. (s.f.). *EAPN Key Documents: Publicaciones e informes sobre pobreza y exclusión social en Europa*. <https://www.eapn.eu/news-and-publications/publications/eapn-key-documents/>

Organización Mundial de la Salud y Banco Mundial. (2011). *Informe mundial sobre la discapacidad*. <https://www.who.int/es/publications/i/item/9789241564182>

Legislación y normativa de accesibilidad

Parlamento Europeo y Consejo de la Unión Europea. (2019). *Directiva (UE) 2019/882 del Parlamento Europeo y del Consejo, de 17 de abril de 2019, sobre los requisitos de accesibilidad de los productos y servicios*. Diario Oficial de la Unión Europea. <https://eur-lex.europa.eu/eli/dir/2019/882/oj>

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. <https://www.w3.org/TR/WCAG22/>

Estudios sobre comunicación aumentativa y pictogramas ARASAAC

Cabello Luque, F., y Mazón Morillas, Y. (2018). *Iconicidad y facilidad de aprendizaje de los símbolos pictográficos de ARASAAC*. Aula Abierta ARASAAC. <https://aulaabierta.arasaac.org/iconicidad-y-facilidad-de-aprendizaje-de-los-simbolos-pictograficos-de-arasaac>

Paolieri, D., y Marful, A. (2018). Norms for name agreement, familiarity, subjective frequency, and imageability for 348 words in European Spanish and Romanian. *Frontiers in Psychology*, 9, 2538. <https://doi.org/10.3389/fpsyg.2018.02538>

Software, frameworks y recursos de código abierto

Asociación para el Desarrollo de los Bancos de Tiempo y Coopdevs. (s.f.). *TimeOverflow* [Plataforma de software]. <https://timeoverflow.org/>

Gobierno de Aragón. (s.f.). *Portal Aragonés de la Comunicación Aumentativa y Alternativa (ARASAAC)*. Centro Aragonés de la Comunicación Aumentativa y Alternativa. <https://arasaac.org/>

Google LLC. (s.f.-a). *Guide to app architecture*. Android Developers. <https://developer.android.com/topic/architecture>

Google LLC. (s.f.-b). *Material Design 3*. <https://m3.material.io/>

Google LLC. (s.f.-c). *Room persistence library*. Android Developers. <https://developer.android.com/training/data-storage/room>

Jahoda, P. (2020). *MPAndroidChart* (Versión 3.1.0) [Biblioteca de software]. GitHub. <https://github.com/PhilJay/MPAndroidChart>

JetBrains s.r.o. (s.f.). *Kotlin Programming Language*. <https://kotlinlang.org/>

Palao, S. (s.f.). *Catálogo de pictogramas ARASAAC*. Centro Aragonés de la Comunicación Aumentativa y Alternativa y Gobierno de Aragón. <https://arasaac.org/pictograms/search>

Plataformas competidoras y contexto del sector

Good Hood GmbH. (s.f.). *Nebenan.de: Die Nachbarschaftsplattform*. <https://nebenan.de/>

Nextdoor Holdings, Inc. (s.f.). *Nextdoor* [Plataforma digital]. <https://nextdoor.es/>

TimeRepublik. (s.f.). *TimeRepublik: The first economy built on the fairness of time*. <https://timerepublik.com/>

Zona Retiro. (2023, 27 de septiembre). *Cierra '¿Tienes sal?': La plataforma vecinal dice adiós por falta de presupuesto*. <https://zonaretiro.com/ciudadanos/cierra-tienes-sal/>

Contexto socioeconómico y aislamiento urbano

Liberal de Castilla. (s.f.). *Aislamiento social en las grandes ciudades: cómo lo están superando las comunidades locales*. <https://www.liberaldecastilla.com/aislamiento-social-en-las-grandes-ciudades-como-lo-estan-superando-las-comunidades-locales>

12. Anexos

12.1. Manual de Instalación

Este manual describe los pasos necesarios para instalar y ejecutar la aplicación VecindApp en un dispositivo Android. Se contemplan dos vías alternativas: la instalación directa del fichero APK empaquetado con la entrega (opción recomendada para el tribunal y para cualquier usuario que únicamente quiera probar la aplicación) y la compilación desde el código fuente en Android Studio (opción dirigida a quien desee inspeccionar o modificar el código).

Requisitos previos

- Dispositivo Android con versión 8.0 (API 26) o superior.
- Aproximadamente 30 MB de espacio libre en el almacenamiento del dispositivo.
- Cable USB compatible con el dispositivo.
- Ordenador con sistema operativo Windows, macOS o Linux.

Solo para la compilación desde el código fuente se requiere además:

- Android Studio Panda 2 (2025.3.2) o superior.
- JDK 11 o superior (incluido con Android Studio).
- Conexión a Internet la primera vez (Gradle descargará las dependencias automáticamente).

Opción A — Instalación directa del APK (la recomendada)

Esta es la forma más rápida de instalar la aplicación y la recomendada para evaluadores y usuarios finales. El fichero APK se entrega dentro del ZIP del código fuente, en la carpeta `app/build/outputs/apk/debug/`.

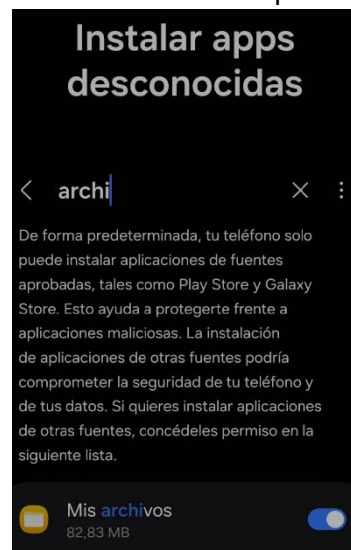
Paso 1. Habilitar la instalación de fuentes desconocidas en el dispositivo

Como el APK no se distribuye a través de Google Play Store, Android requiere habilitar explícitamente la instalación de aplicaciones desde otras fuentes. El procedimiento varía ligeramente según la versión de Android y el fabricante, pero en general:

1. Abrir la aplicación Ajustes del dispositivo.

2. Acceder a Seguridad o Privacidad (según el fabricante).
3. Activar la opción Fuentes desconocidas o Instalar aplicaciones desconocidas para la aplicación desde la que se vaya a abrir el APK (por ejemplo, el explorador de archivos o el navegador).

Captura 18. Ajustes de seguridad del dispositivo con la opción de fuentes desconocidas.



Paso 2. Transferir el APK al dispositivo

Existen varias formas de hacer llegar el fichero APK al dispositivo. Las más cómodas son:

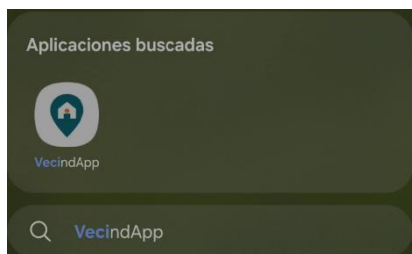
- **Por cable USB:** conectar el dispositivo al ordenador mediante el cable USB, aceptar el permiso de transferencia de archivos en el dispositivo y copiar el fichero `app-debug.apk` a la carpeta de Descargas del móvil.
- **Por correo electrónico:** enviarse el APK como adjunto a una cuenta de correo accesible desde el móvil y descargarlo desde ahí.
- **Por servicios de nube:** subir el APK a Google Drive, Dropbox o similar y descargarlo desde el dispositivo.

Paso 3. Instalar la aplicación

1. Abrir el explorador de archivos del dispositivo.
2. Localizar el fichero `VecindApp.apk` en la carpeta donde se haya descargado.
3. Pulsar sobre el fichero para iniciar la instalación.
4. Aceptar los permisos solicitados cuando Android muestre el diálogo de confirmación.
5. Esperar unos segundos a que se complete la instalación.

Paso 4. Abrir la aplicación

Una vez instalada, VecindApp aparecerá en el cajón de aplicaciones del dispositivo con su icono característico. Al abrirla por primera vez, se muestra la pantalla de Login y la base de datos local SQLite se crea automáticamente con dos usuarios de prueba precargados mediante `SeedDatabaseCallback`, lo que permite probar el flujo multiusuario sin necesidad de registrar nuevos vecinos.



Captura 19. Icono de VecindApp en el cajón de aplicaciones.

Opción B — Compilación desde el código fuente

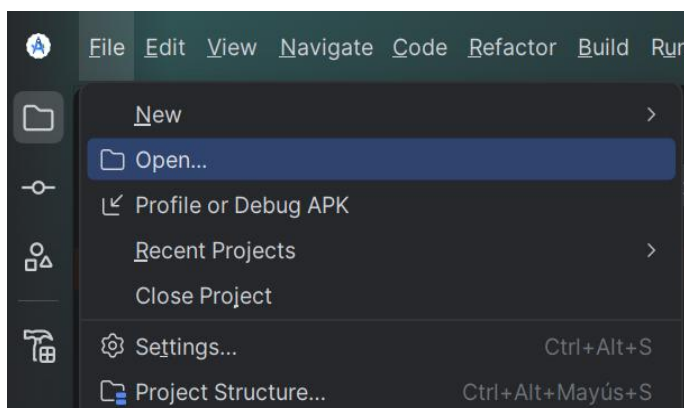
Esta vía está dirigida a quien quiera inspeccionar el código, compilar el proyecto por su cuenta o modificar la aplicación.

Paso 1. Descomprimir el código fuente

El código fuente se entrega como un fichero ZIP. Descomprimirlo en una carpeta local del ordenador. Se generará una estructura con el proyecto Gradle completo, incluyendo los ficheros `build.gradle.kts`, `settings.gradle.kts`, `libs.versions.toml` y la carpeta `app/` con el código de la aplicación.

Paso 2. Abrir el proyecto en Android Studio

1. Iniciar Android Studio.
2. Seleccionar Open en la pantalla de bienvenida (o File → Open si ya se encuentra en un proyecto distinto).
3. Navegar hasta la carpeta descomprimida en el paso anterior y seleccionarla.
4. Pulsar OK.



Captura 20. Opción en Android Studio con la opción Open resaltada.

Paso 3. Sincronización de Gradle

La primera vez que se abra el proyecto, Android Studio ejecutará automáticamente una sincronización

de Gradle que descargará todas las dependencias declaradas en `libs.versions.toml`: Room, Navigation Component, ViewModel + LiveData, MPAndroidChart, Material Components y el resto de librerías. Este proceso puede tardar varios minutos la primera vez, dependiendo de la velocidad de la conexión a Internet. Al finalizar, aparecerá el mensaje BUILD SUCCESSFUL en la pestaña Build de la parte inferior.

Paso 4. Conectar el dispositivo físico

1. En el dispositivo Android, habilitar las Opciones de desarrollador. Para ello, acceder a Ajustes → Acerca del teléfono y pulsar 7 veces sobre el Número de compilación. Aparecerá un mensaje confirmando que las opciones de desarrollador han sido activadas.
2. Acceder a la nueva sección Ajustes → Opciones de desarrollador y activar la Depuración USB.
3. Conectar el dispositivo al ordenador mediante el cable USB.
4. Aceptar en el dispositivo el diálogo que solicita permiso de depuración USB desde ese ordenador (marcando la casilla "Permitir siempre" para no tener que repetir la autorización en futuras conexiones).

Paso 5. Compilar e instalar en el dispositivo

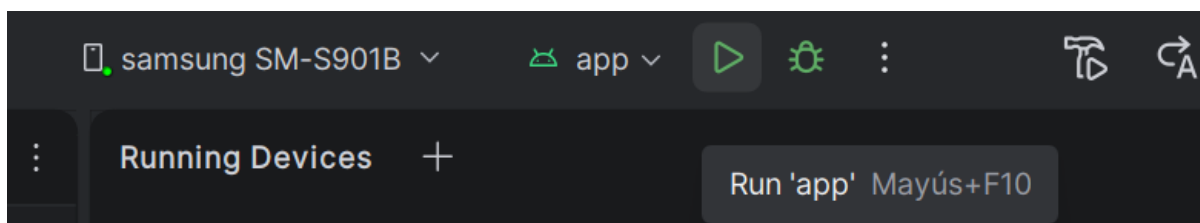
Una vez conectado y detectado el dispositivo, aparecerá en la lista desplegable de dispositivos de destino en la parte superior de Android Studio. Para compilar e instalar:

1. Seleccionar el dispositivo en la lista desplegable.
2. Pulsar el botón Run 'app' (icono de triángulo verde) o utilizar el atajo Shift + F10.

Android Studio compilará el proyecto, generará el APK, lo instalará automáticamente en el dispositivo y abrirá la aplicación. Alternativamente, desde la línea de comandos (terminal integrada de Android Studio o consola externa ubicada en la raíz del proyecto) se pueden ejecutar directamente las tareas Gradle:

```
./gradlew clean assembleDebug # Genera el APK
```

```
./gradlew installDebug # Instala el APK en el dispositivo conectado
```



Botón Run 'app' de Android Studio con el dispositivo físico seleccionado.

Paso 6. Verificación del funcionamiento

Al abrirse la aplicación por primera vez tras la instalación, debe aparecer la pantalla de Login con los dos usuarios de prueba precargados en la base de datos. Si este es el caso, la instalación se ha realizado correctamente.

Resolución de problemas frecuentes

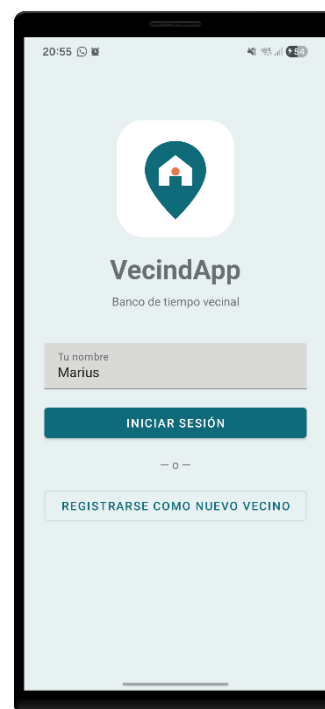
- **El APK no se instala y muestra "Aplicación no instalada".** Comprobar que el dispositivo cumple el requisito mínimo de Android 8.0 (API 26) y que la opción de fuentes desconocidas está habilitada correctamente para la aplicación desde la que se intenta abrir el APK.
- **Android Studio no detecta el dispositivo conectado por USB.** Verificar que la depuración USB está habilitada, que el cable permite transferencia de datos (algunos cables solo cargan) y que se ha aceptado el diálogo de autorización en el dispositivo. En Windows puede ser necesario instalar los drivers USB específicos del fabricante del dispositivo.
- **Gradle falla durante la sincronización con errores de descarga de dependencias.** Comprobar la conexión a Internet y reintentar la sincronización (File → Sync Project with Gradle Files). En caso de trabajar en entornos corporativos con firewalls que intercepten HTTPS pueden requerirse ajustes adicionales; ante este tipo de incidencias, contactar con el autor.
- **La aplicación se abre pero muestra un listado vacío en el escaparate.** Esta situación es posible si la base de datos se ha creado sin los usuarios de prueba. Desinstalar la aplicación desde el dispositivo y volver a instalarla para que SeedDatabaseCallback vuelva a ejecutarse.

12.2. Manual de Usuario

Este manual describe las funcionalidades principales de VecindApp desde el punto de vista del usuario final. Se presenta siguiendo el flujo natural de uso de la aplicación: desde el primer acceso y registro, pasando por la publicación y solicitud de servicios, hasta la valoración final y la consulta del historial de actividad. Cada apartado incluye capturas de pantalla de referencia.

Primer acceso a la aplicación

Al abrir VecindApp por primera vez, aparecerá la pantalla de Login. Aquí el usuario dispone de dos opciones: introducir el nombre de un vecino ya registrado para iniciar sesión, o pulsar el enlace inferior "Registrarse como nuevo vecino" para crear una cuenta nueva.



La aplicación viene precargada con dos usuarios de prueba para facilitar la demostración multiusuario. Los nombres exactos aparecen en la propia base de datos tras la primera ejecución.

Registro de un nuevo vecino

Al pulsar "Registrarse como nuevo vecino", se accede al formulario de registro. Este formulario solicita dos datos:

- **Nombre:** el nombre con el que el vecino será visible en la comunidad. Debe ser único (la aplicación impide registrar dos vecinos con el mismo nombre).
- **Barrio:** se selecciona de un desplegable con los 36 barrios de Zaragoza predefinidos.

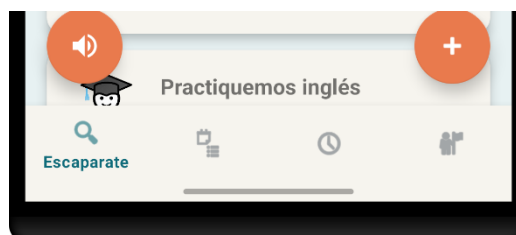
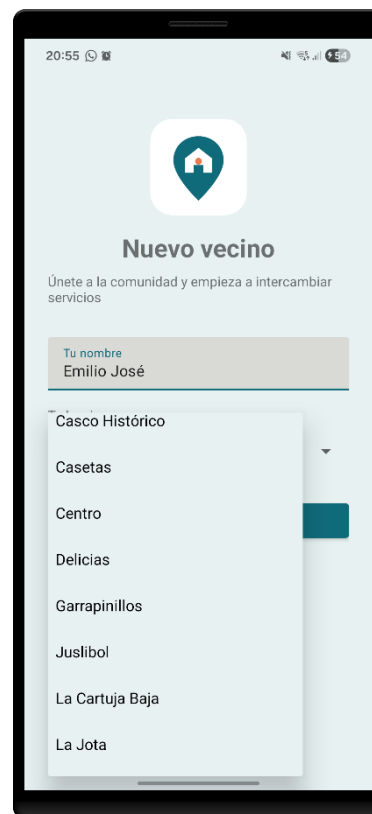
Una vez completados los campos, el botón "Registrarse" crea la cuenta, asigna un saldo inicial de bienvenida de 5 horas e inicia automáticamente la sesión del nuevo vecino, llevándolo al Escaparate.

Navegación principal: el BottomNav

Una vez iniciada la sesión, la aplicación presenta cuatro pestañas accesibles desde la barra inferior:

- **Escaparate:** listado de todos los servicios activos publicados por la comunidad.
- **Transacciones:** gestión de las transacciones activas del usuario (pendientes, aceptadas).
- **Historial:** consulta de las transacciones ya completadas o canceladas, con gráfico de balance de horas.
- **Perfil:** información personal del vecino (saldo, nivel, servicios publicados) y opción de cerrar sesión.

En la pestaña Transacciones puede aparecer un badge numérico rojo que indica cuántas transacciones requieren atención del usuario (pendientes de aceptar o completadas sin valorar).



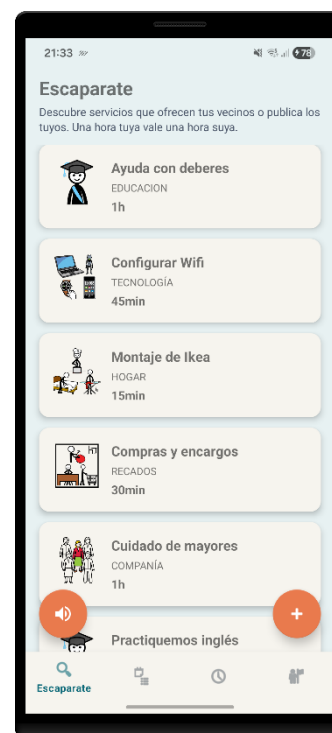
Explorar el Escaparate

El Escaparate muestra todos los servicios activos de la comunidad ordenados por fecha de publicación más reciente. Cada tarjeta muestra el título del servicio, su coste en horas y un pictograma ARASAAC asignado según su categoría (Hogar, Tecnología, Educación, Compañía, Recados u Otros).

Dos botones flotantes (FABs) ayudan en la navegación:

- El FAB con el icono "+" (color naranja) permite publicar un nuevo servicio.
- El FAB con el icono de altavoz (color naranja) activa el Text-to-Speech y lee en voz alta los servicios visibles en pantalla.

Adicionalmente, mantener pulsada una tarjeta específica activa el TTS solo para ese servicio concreto.

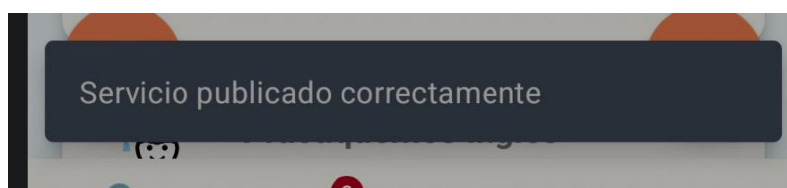


Publicar un servicio

Al pulsar el FAB "+", se accede al formulario de creación de servicio. Los campos a rellenar son:

- **Título:** nombre corto del servicio (ejemplo: "Ayuda con el ordenador").
- **Descripción:** detalles adicionales opcionales sobre el servicio.
- **Categoría:** selección en un desplegable de una de las 6 categorías disponibles.
- **Coste en horas:** se ajusta mediante un control deslizante (Slider) con pasos de 15 minutos (0,25 h) y un rango de 0,25 a 8 horas. Mientras se arrastra, aparece una etiqueta flotante con el valor actual en formato humanizado (por ejemplo, "1h 30min").

Al pulsar "Guardar", el servicio queda publicado y visible inmediatamente en el Escaparate de todos los vecinos.



Detalle de un servicio y acciones disponibles

Al pulsar cualquier tarjeta del Escaparate, se accede a la pantalla de Detalle del Servicio. Esta pantalla muestra toda la información del servicio: título, descripción completa, categoría, coste y estado actual.

Los botones de acción disponibles dependen de quién sea el usuario activo y del estado del servicio:

- Si el servicio es propio y está en estado **ACTIVO**: aparecen los botones "Editar" y "Eliminar".
- Si el servicio es de otro vecino y está en estado **ACTIVO**: aparece el botón "Solicitar servicio".
- Si el servicio está en estado **RESERVADO** o **COMPLETADO**: no se mostrará en el escaparate. Podrás ver en pestaña "Transacciones"; el servicio sólo si lo has solicitado tu o eres quien lo ha publicado

Un botón de altavoz en la esquina inferior izquierda permite escuchar la información del servicio mediante TTS.



Detalle del servicio desde el punto de vista del propietario (botones Editar/Eliminar).

Detalle del servicio desde el punto de vista de otro vecino (botón Solicitar servicio)

Solicitar un servicio

Cuando un vecino pulsa "Solicitar servicio" en el detalle de un servicio ajeno, el sistema verifica automáticamente que su saldo de horas sea suficiente para cubrir el coste. Si es así, aparece un diálogo de confirmación que muestra las horas que se transferirán al completarse la transacción.

la transacción.

Al confirmar, la transacción queda creada en estado **PENDIENTE** y el servicio cambia a estado **RESERVADO**, desapareciendo temporalmente del Escaparate hasta que se complete o se cancele.

Si el saldo es insuficiente, la aplicación lo indica con un mensaje emergente (Snackbar) y no bloquea el servicio, permitiendo que siga siendo visible para otros vecinos.

Gestionar las transacciones activas

La pestaña Transacciones muestra todas las transacciones activas del usuario, tanto aquellas en las que actúa como comprador (chip azul) como aquellas en las que actúa como vendedor (chip verde). Esta diferenciación visual facilita identificar de un vistazo el rol que se desempeña en cada transacción.

Los botones disponibles en cada tarjeta dependen del rol y del estado:

- **Vendedor, estado PENDIENTE:** botones "Aceptar" y "Cancelar".
- **Vendedor, estado ACEPTADA:** botón "Completar" y "Cancelar".
- **Comprador, estado PENDIENTE o ACEPTADA:** botón "Cancelar".
- **Comprador, estado COMPLETADA:** botón "Valorar".



Todas las acciones destructivas (Cancelar, Completar) muestran un diálogo de confirmación antes de ejecutarse, para evitar cambios accidentales en el saldo de horas.

Completar una transacción

Cuando un vendedor pulsa "Completar" y confirma, el sistema ejecuta automáticamente y en una sola operación:

- Resta las horas acordadas al comprador.
- Suma las horas acordadas al vendedor.
- Incrementa el contador de intercambios de ambos y recalcula su nivel de vecino.

- Marca la transacción como COMPLETADA.

Desde este momento, el comprador verá un nuevo botón "Valorar" en su pestaña de Transacciones y un badge numérico en el BottomNav que le recordará que tiene una valoración pendiente de emitir.

Valorar con pictogramas ARASAAC

Una de las características diferenciadoras de VecindApp es su sistema de valoración mediante pictogramas ARASAAC en lugar de estrellas numéricas. Cuando un comprador pulsa "Valorar" tras completarse una transacción, se abre un panel inferior (BottomSheet) con 9 pictogramas seleccionables que representan cualidades del vendedor (puntual, amable, cumple lo prometido, muy recomendable, etc.).

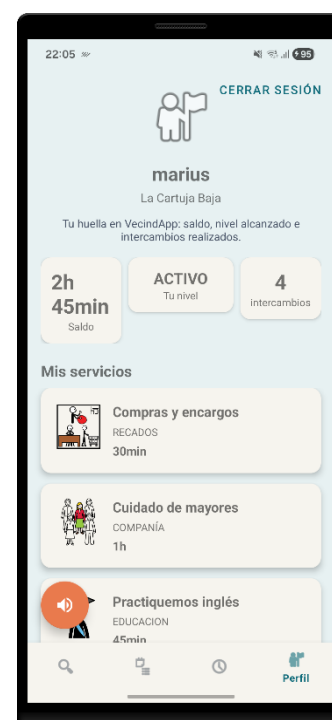
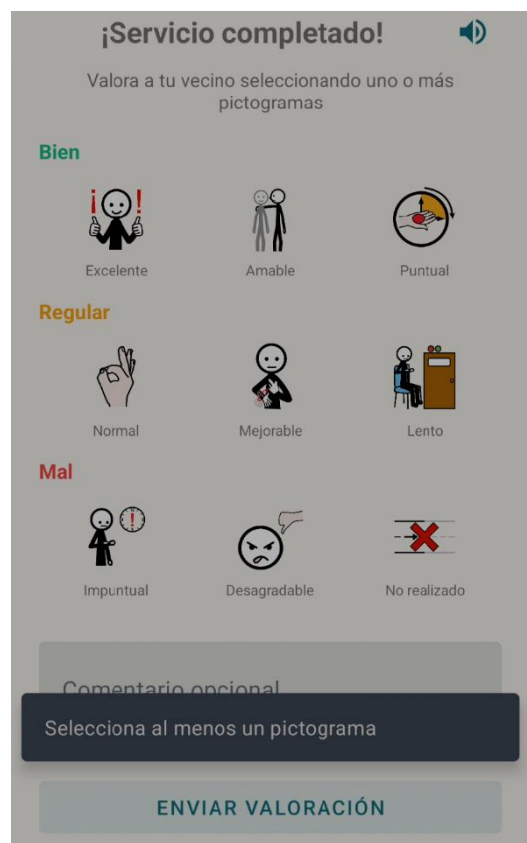
El comprador puede seleccionar uno o varios pictogramas y, opcionalmente, añadir un comentario de texto libre. Un botón de altavoz permite escuchar por TTS los pictogramas seleccionados y el comentario antes de enviarla, facilitando la revisión a usuarios con dificultades de lectura.

Al pulsar "Enviar valoración", la valoración queda guardada y asociada a la transacción.

Consultar el perfil

La pestaña Perfil muestra la información personal del vecino:

- Nombre y barrio.
- Saldo actual de horas disponibles.
- Nivel del vecino (Novato, Activo, Veterano o Referente) calculado automáticamente según el número de intercambios completados.
- Contador total de intercambios realizados.



- Lista de Mis servicios: todos los servicios publicados por el usuario, que se pueden pulsar para acceder a su detalle.

Desde esta pestaña también se accede al botón "Cerrar sesión", que limpia la sesión actual y devuelve al usuario a la pantalla de Login.

Un botón de altavoz en la parte superior permite escuchar por TTS los datos del perfil.

Consultar el historial

La pestaña Historial permite revisar toda la actividad pasada del vecino en la aplicación. Está organizada en dos secciones:

- Un gráfico de barras agrupadas en la parte superior (generado con MPAndroidChart) que muestra el balance mensual de horas ganadas frente a horas gastadas.
- Dos pestañas horizontales debajo del gráfico: una para las transacciones completadas y otra para las canceladas.

Las transacciones completadas se muestran con el signo apropiado: +N horas (en verde) si el usuario ganó horas como vendedor, o -N horas (en rojo) si las gastó como comprador. Las canceladas aparecen atenuadas, con el texto tachado y la leyenda "Sin cargo", indicando que no hubo transferencia de horas.



Al pulsar cualquier tarjeta, se accede al detalle del servicio asociado; desde ese detalle, si la transacción tiene valoración, un botón permite consultarla en un BottomSheet modal.

Uso del Text-to-Speech (TTS)

VecindApp incorpora la síntesis de voz en las pantallas principales (Escaparate, Detalle del Servicio, Perfil y BottomSheet de Valoración) como ayuda a la accesibilidad. El botón de altavoz está siempre visible en estas pantallas y no requiere que el usuario active ningún servicio externo de Android.

La lectura se adapta al contenido de cada pantalla: en el Escaparate se leen los servicios visibles uno por uno, en el Detalle se leen las etiquetas y los valores del servicio, en el Perfil se lee el resumen de la actividad del usuario y en la Valoración se leen los pictogramas seleccionados junto con el comentario.

La lectura se detiene automáticamente al cambiar de pantalla o al salir de la aplicación, evitando superposiciones o lecturas en segundo plano.

Cerrar sesión

Al pulsar el botón "**Cerrar sesión**" desde la pantalla de Perfil, el sistema finaliza la sesión activa, limpia el badge de notificaciones y regresa a la pantalla de Login. La información del vecino permanece guardada en la base de datos local y puede recuperarse iniciando sesión nuevamente con el mismo nombre.



12.3. Diagramas

Casos de uso

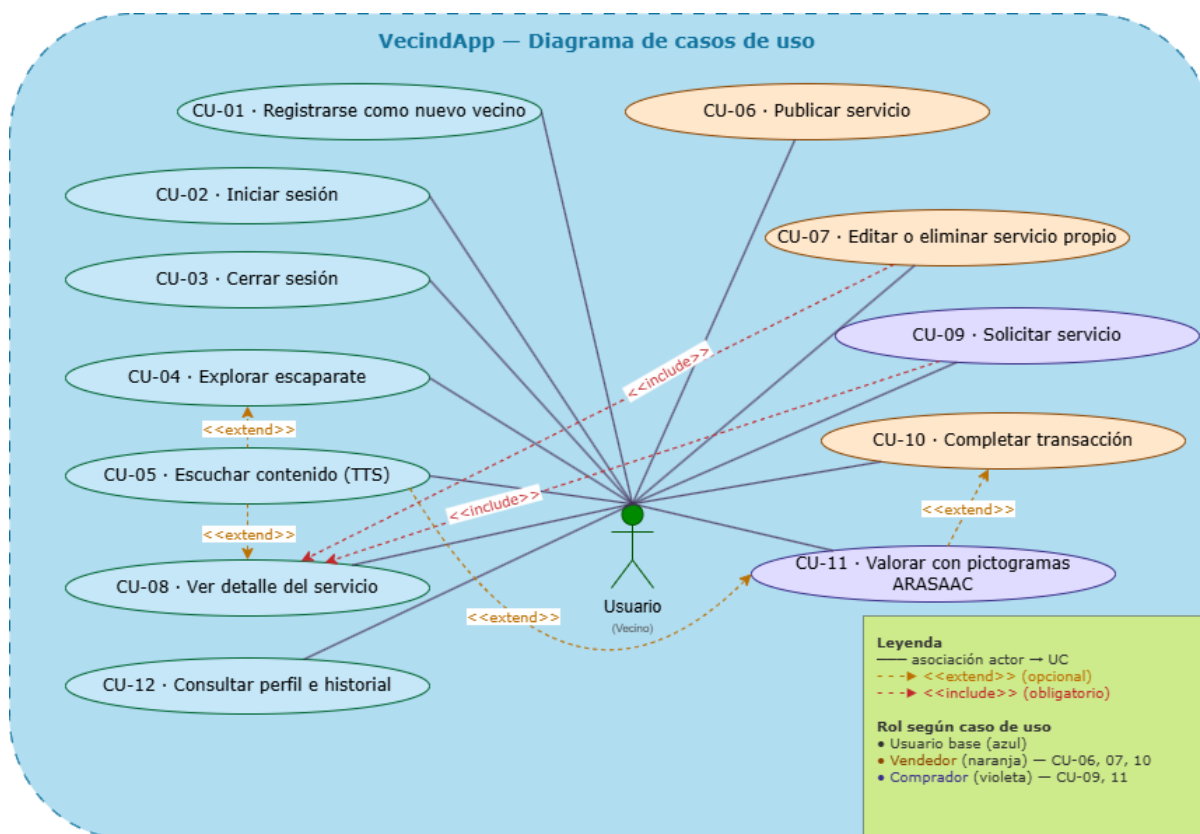
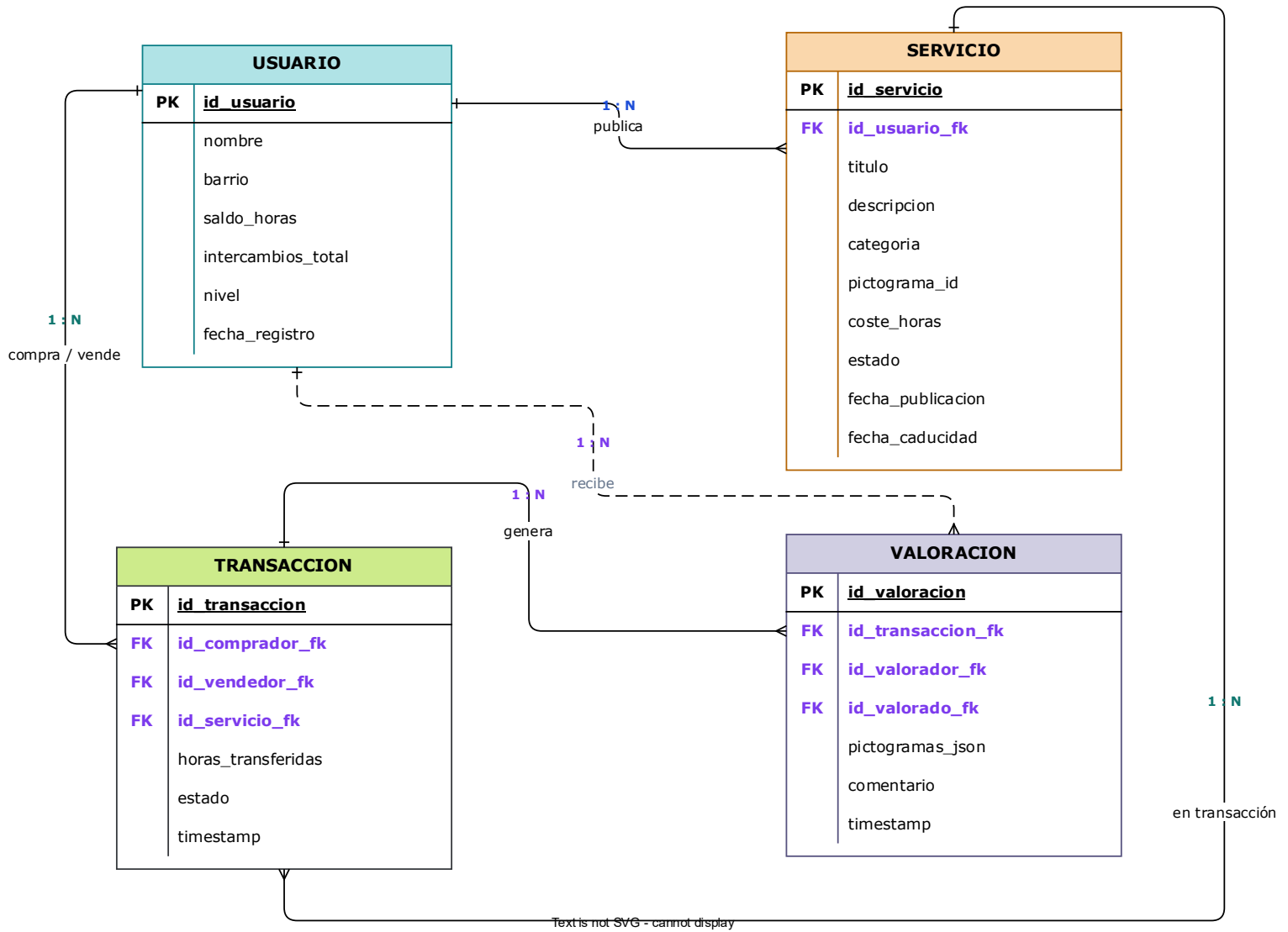
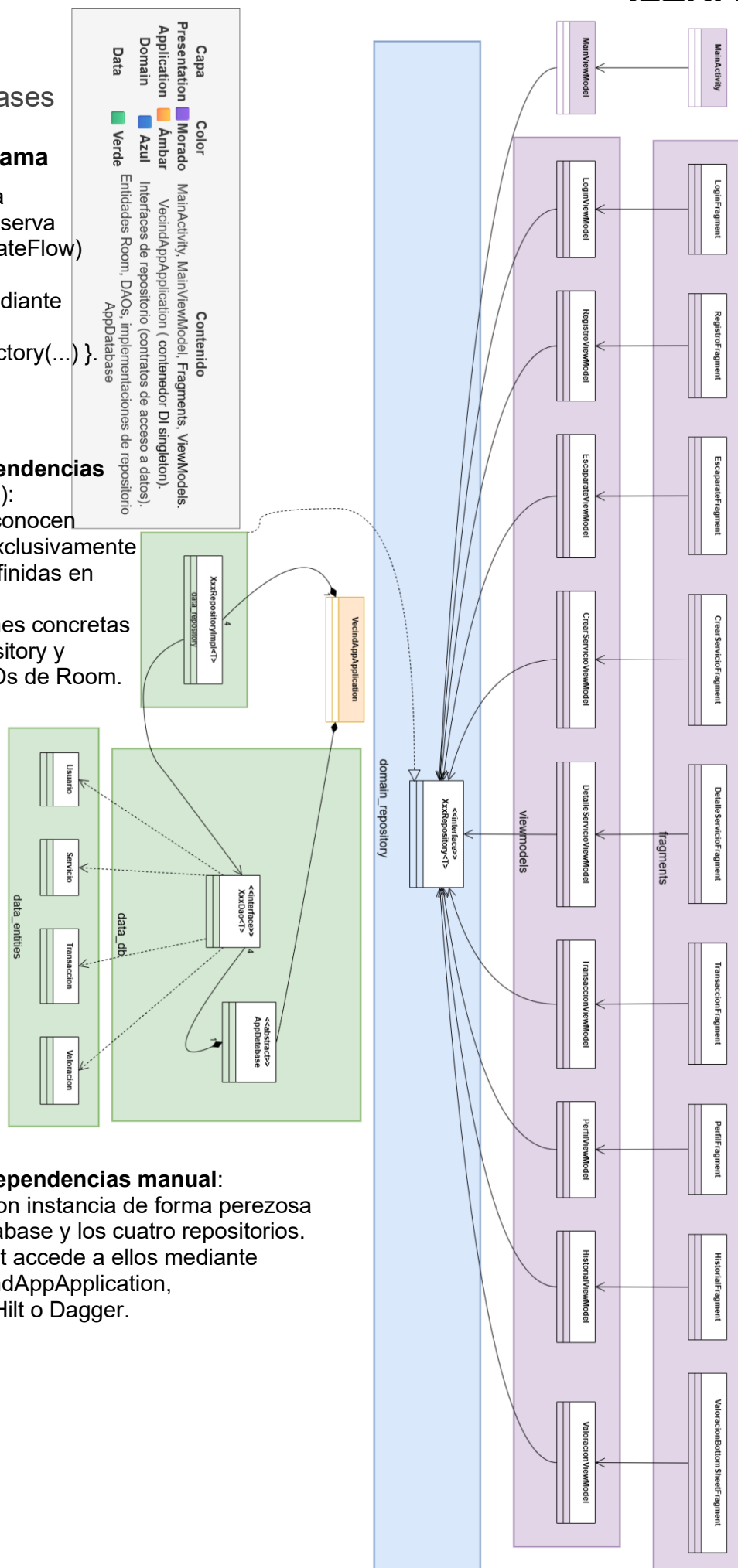
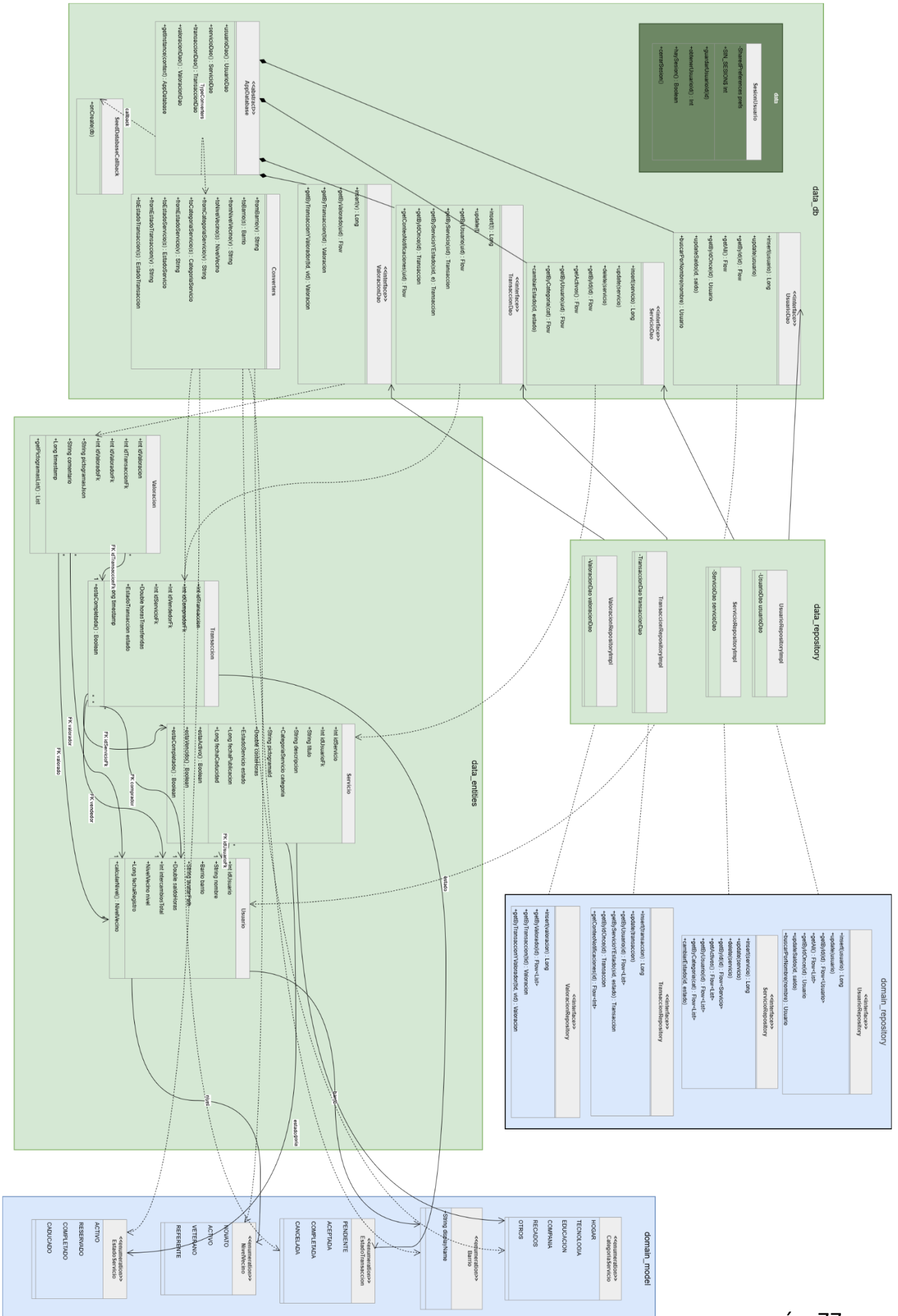


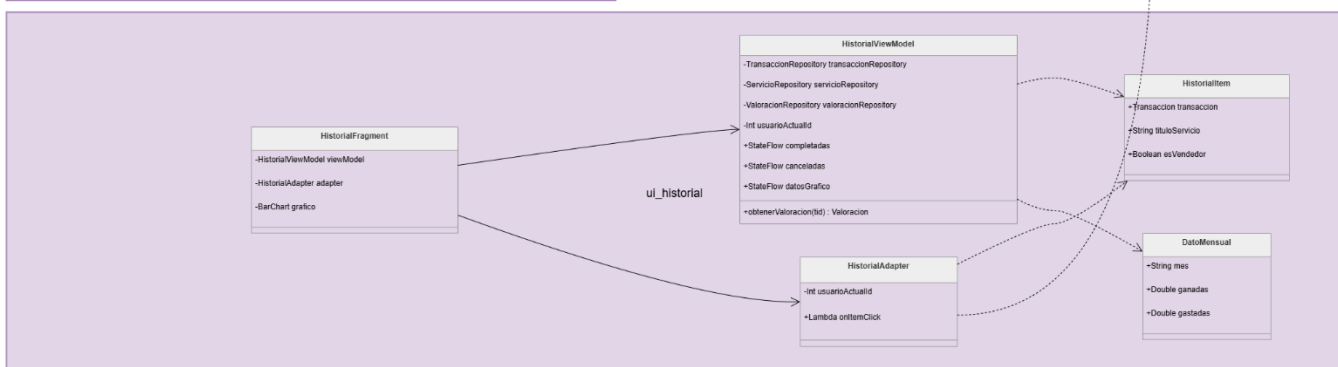
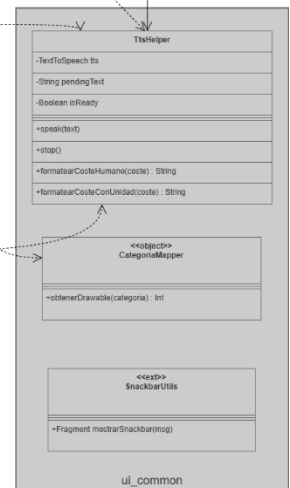
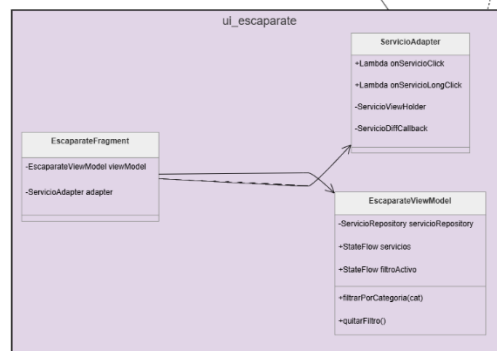
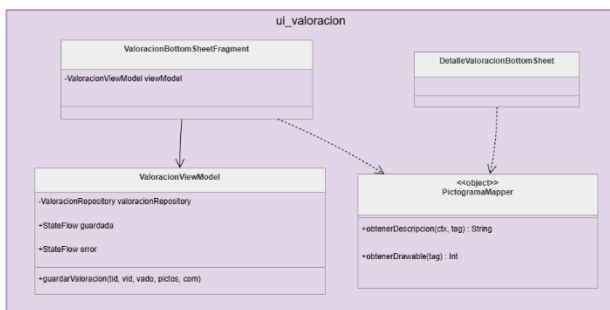
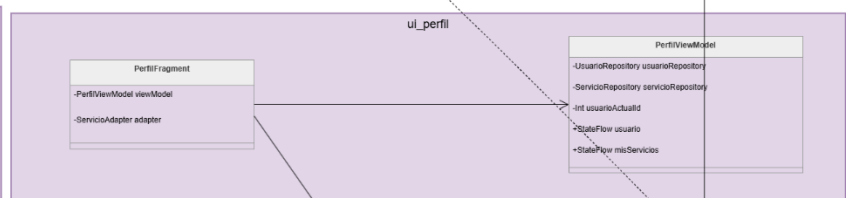
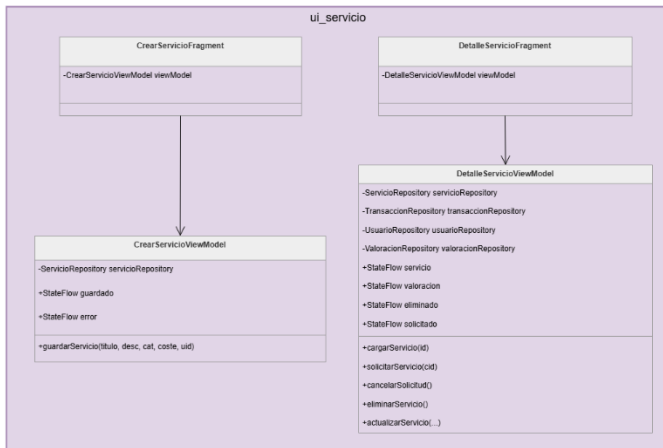
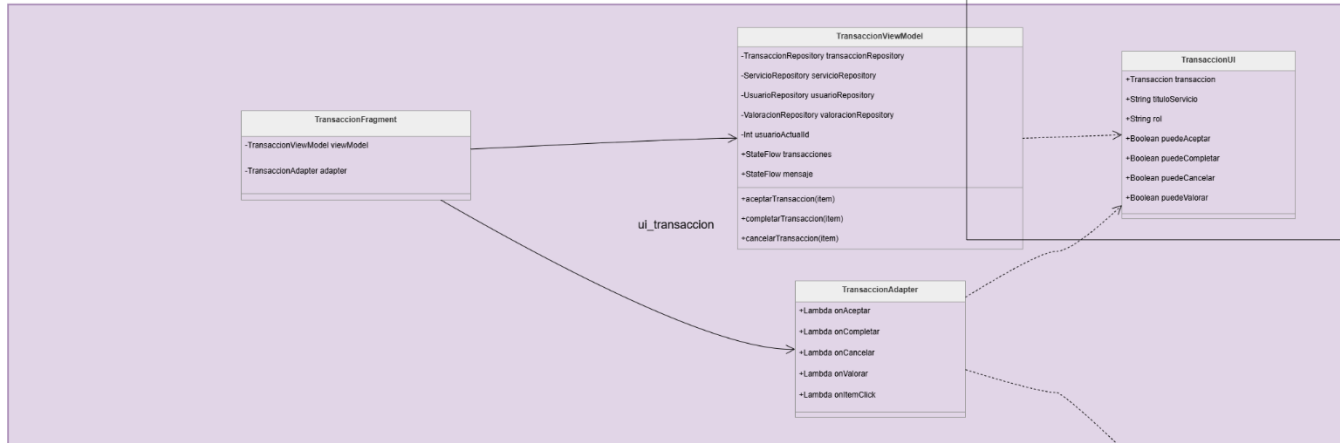
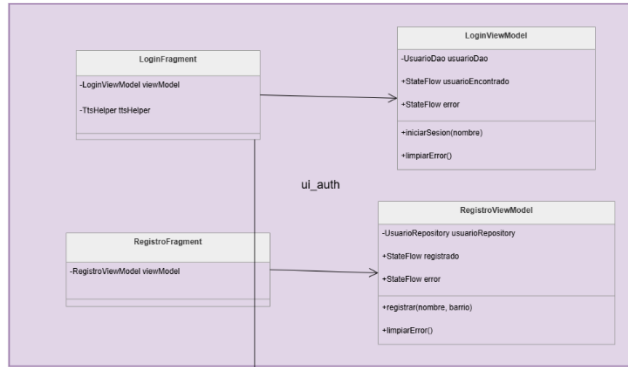
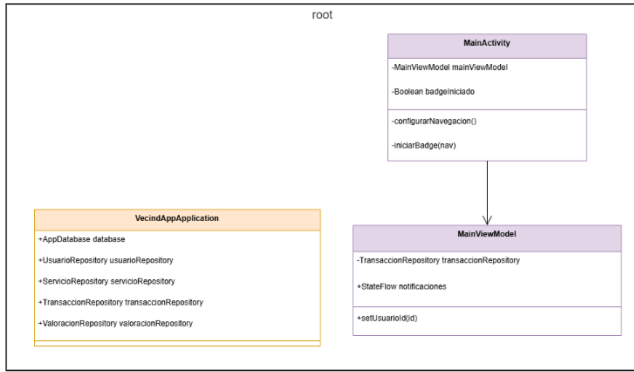
Diagrama Entidad-Relación







VecindApp



12.4. Glosario

MVVM (Model-View-ViewModel) es un patrón de arquitectura de software utilizado principalmente en el desarrollo de aplicaciones con interfaces gráficas, especialmente en entornos como Android, iOS y aplicaciones de escritorio. Su objetivo es separar la lógica de negocio y la gestión de datos (Model), la presentación visual (View) y la lógica de presentación o interfaz (ViewModel). El ViewModel actúa como intermediario entre el modelo y la vista, facilitando la vinculación de datos y permitiendo una mayor escalabilidad, mantenibilidad y testabilidad del código.

FAB : El Floating Action Button (FAB) es un botón circular destacado que generalmente se coloca en la esquina inferior derecha de la interfaz de usuario en aplicaciones móviles. Su propósito principal es resaltar y facilitar el acceso rápido a una acción principal o más importante dentro de una pantalla, como añadir un nuevo elemento, crear contenido o iniciar una tarea esencial.

